European Journal of Operational Research 289 (2021) 470–484

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

The Pickup and Delivery Problem with Split Loads and Transshipments: A Branch-and-Cut Solution Approach

David Wolfinger[a,*], Juan-José Salazar-González[b]

[a] Christian Doppler Laboratory for Efficient Intermodal Transport Operations, Department of Business Decisions and Analytics, University of Vienna, Oskar-Morgenstern-Platz 1, 1090 Vienna, Austria
[b] Department of Mathematics, Statistics and Operations Research, University of La Laguna, 38200 La Laguna, Tenerife, Spain

ARTICLE INFO

Article history:
Received 18 December 2018
Accepted 19 July 2020
Available online 23 July 2020

Keywords:
Routing
Pickup and delivery
Split loads
Transshipments
Branch-and-cut

ABSTRACT

This paper addresses the design of routes for a fleet of vehicles serving customer requests. It is a pickup and delivery problem where each customer location can be visited several times, and load may be transshipped from one vehicle to another at specific locations. The aim is to minimize the sum of travel costs and transshipment costs. We present an arc-based mixed-integer formulation for the problem and propose a branch-and-cut algorithm for solving it. Extensive computational results illustrate the performance of the algorithm.

© 2020 The Author(s). Published by Elsevier B.V.
This is an open access article under the CC BY license. (http://creativecommons.org/licenses/by/4.0/)

1. Introduction

In the Pickup and Delivery Problem (PDP) a fleet of capacitated vehicles must be routed in order to satisfy a set of customer requests. A request consists of picking up a load at an origin location and delivering it to a destination location. Several requests may be transported simultaneously by the same vehicle as long as the combined load does not exceed the vehicle capacity. Each origin and destination must be visited exactly once. The vehicle routes must start and finish at the vehicle depots, and they must satisfy precedence and coupling constraints: for each request, the origin must be visited before the destination, and both locations must be visited by the same vehicle. The aim is to find a set of vehicle routes such that the transportation costs are minimized. The PDP is an extension of the well-known Capacitated Vehicle Routing Problem (CVRP) and has a wide range of practical applications in both people and freight transportation (see Toth & Vigo, 2014). The PDP has been extensively studied over the past decades and we refer the interested reader to Berbeglia, Cordeau, Gribkovskaia, and Laporte (2007) and Parragh, Doerner, and Hartl (2008) for detailed reviews.

In this article, we address a generalized version of the PDP, called the Pickup and Delivery Problem with Split Loads and Transshipments (PDPSLT). We present an arc-based mixed-integer formulation for the PDPSLT and propose a branch-and-cut algorithm for solving it.

In the PDPSLT the constraint that each customer location must be visited exactly once is relaxed, assuming that the request is splitable. This freedom allows for the total load of a request to be delivered in several trips rather than only one. Although it might be counter-intuitive at first, performing more visits to a location may indeed result in lower costs because of the vehicle capacity limitations. Splitting a load can potentially eliminate a dedicated trip for delivery by allocating partial loads to other vehicles which have excess capacity. Additionally, in the PDPSLT, load may be transshipped from one vehicle to another at intermediary locations, called *transshipment locations*. Depending on the application, transshipment may take place either at customer locations (other than the request's origin and destination) or at dedicated transshipment locations, or both. This article considers the last case. Allowing transshipment requires the relaxation of the coupling constraint since the vehicle which picks up the load from its origin may not be the same vehicle which delivers it to its destination. Furthermore, allowing transshipment implies a new precedence constraint which states that if a load uses a transshipment location, it must first be delivered at the transshipment location by the first vehicle before the second vehicle can pick it up. Similar to splitting loads, transshipping loads may lead to lower costs through better utilization of vehicle capacities.

The PDPSLT arises naturally in the context of multimodal transportation, where transshipments need to take place per definition. In multimodal transportation, whenever goods are not transported

* Corresponding author
E-mail addresses: david.wolfinger@univie.ac.at (D. Wolfinger), jjsalaza@ull.es (J.-J. Salazar-González).

https://doi.org/10.1016/j.ejor.2020.07.032
0377-2217/© 2020 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license. (http://creativecommons.org/licenses/by/4.0/)

in containers, requests are often splitable as well. This is the case, for example, for raw materials or steel products (either unpacked or packed in smaller transportation units such as bags, barrels, pallets, crates or boxes). According to EUROSTAT (the Statistical Office of the European Union),[1] the share of containerized transport by rail in the EU was 17.9% in 2017. For inland waterway transport the share of containerized transport was only 9.2%. Given that most companies in the EU do not have a direct connection to a rail network and are not located at a port, it is safe to assume that the majority of rail transport, respectively inland waterway transport, is part of a multimodal transport chain. Therefore, based on the above numbers, there is a large potential for applying the PDPSLT in multimodal transportation. The PDPSLT also lends itself to various other practical applications such as door-to-door transportation services for people and less-than-truckload transportation. In door-to-door people transportation, transshipments and split loads are already both studied in the literature and used in some real-world cases, albeit separately (see for example Masson, Lehuédé, and Péton (2013) for transshipments or Parragh, Pinho de Sousa, and Almada-Lobo (2015) for split loads). Combining both in order to further decrease costs would be the logically next step. Likewise, in less-than-truckload transportation, splitting load is already considered in some real-world cases as well as addressed in the literature (see for example Nowak, Ergun, & White III, 2008). Depending on the type of transported goods and the available infrastructure, transshipping requests can often additionally be considered.

Despite its wide-ranging practical applicability, the PDPSLT has received little attention in the scientific literature. In fact, the only closely-related work we are aware of is by Kerivin, Lacroix, Mahjoub, and Quilliot (2008). They assume that all vehicles are identical, that there are no transshipment costs, and that there is a completion time limit $\mathcal{T}$ for each vehicle. These assumptions allow to represent a PDPSLT solution as paths in an acyclic time-space graph (called auxiliary graph in their article and with $\mathcal{T}+1$ vertices for each location). A time-space graph helps imposing precedence constraints to guarantee that the collection of a request is done before its delivery. Another advantage is that it does not require to know an upper bound on the number of visits to a location by a vehicle. However, a clear disadvantage is that it is an impractical representation when the time period cannot be discretized into very few time instants. Contrary to our study, they do not consider dedicated transshipment locations. Kerivin et al. (2008) propose a branch-and-cut algorithm for solving an arc-based formulation on the time-space graph. Their implementation solved instances with travel times of 1 for all arcs, a completion time limit $\mathcal{T}=7$, and with up to 10 locations. Like them, we do not assume any limit on the number of transshipments per request or at a location. But in contrast to them, we aim at solving PDPSLT instances with no completion time limit (although our model could be modified to include it), with transshipment costs, and with a heterogeneous fleet of vehicles (for example required in multimodal transportation). We only assume a limit on the maximum number of visits of a vehicle to a location, which is a realistic assumption in practice.

Several other different problems which share characteristics with the PDPSLT have been studied in the literature. The option of serving a request in more than one trip has first been studied by Dror and Trudeau (1989, 1990) in the context of the classical CVRP. Their proposed problem relaxes the CVRP and is usually referred to as Split Delivery Vehicle Routing Problem (SDVRP). Since then, the SDVRP has been addressed –and extended (e.g., with time windows)– by several authors (e.g., Archetti, Bianchessi, & Speranza, 2011a; Archetti, Bouchard, & Desaulniers, 2011b; Desaulniers, 2010; Dror, Laporte, & Trudeau, 1994; Gendreau, Dejax, Feillet, & Gueguen, 2006). The most relevant extension of the SDVRP with respect to the PDPSLT is the Split Pickup SDVRP (SPSDVRP) where each customer has a demand of a single commodity, either to be picked up or delivered, and each customer demand can be split. In a recent study Casazza, Ceselli, and Calvo (2020) propose a branch-and-price-and-cut algorithm for the SPSDVRP that manages to solve instances with up to 20 customers. For a detailed review on the SDVRP (and its variants) we refer the reader to the surveys of Archetti and Speranza (2008, 2012).

Nowak et al. (2008) introduce the PDP with split loads (PDPSL). They show that it is NP-hard and present a local search based heuristic for the single vehicle version. They also show that the largest benefit from load splits can be obtained when the size of the loads is just above half of the vehicle capacities. Nowak, Ergun, and White III (2009) present an empirical study on the benefit of split loads in the context of the PDP. Şahin, Çavuşlar, Öncan, Şahin, and Aksu (2013) consider the multi vehicle version of the problem. They also propose a local search based heuristic. Haddad et al. (2018) propose an iterated local search metaheuristic for the same problem. Regarding exact methods, Haddad et al. (2018) additionally present a branch-and-price algorithm for the PDPSL. Stålhane, Andersson, Christiansen, Cordeau, and Desaulniers (2012) solve the maritime version of the PDPSL with time windows by means of a branch-and-price-and-cut method. In the context of people transportation, Parragh et al. (2015) introduce the Dial-a-Ride Problem with split requests and profits. They propose a branch-and-price algorithm as well as a variable neighbourhood search algorithm.

Mitrovic-Minic and Laporte (2006) introduce the PDP with transshipments (PDPT). They show the usefulness of performing transshipments and present a two-phase heuristic to solve the problem. Qu and Bard (2012) solve the PDPT using a GRASP metaheuristic approach which uses an Adaptive Large Neighbourhood Search (ALNS) in the improvement phase. Masson et al. (2013) use a pure ALNS to tackle the PDPT for passenger transportation. They extend their solution method and apply it to the Dial-a-Ride Problem with Transfers in Masson, Lehuédé, and Péton (2014a). Neves-Moreira, Amorim, Guimarães, and Almada-Lobo (2016) propose a fix-and-optimize matheuristic algorithm for the full truckload version of the PDPT. Bouros, Sacharidis, Dalamagas, and Sellis (2011) address the dynamic version of the PDPT. The authors propose a novel heuristic solution method which is based on finding the shortest path from the origin to the destination for each new request. Regarding exact methods, Cortés, Matamala, and Contardo (2010) present an arc-based mixed-integer formulation for the PDPT which they solve by means of a branch-and-cut method using Benders Decomposition. Rais, Alvelos, and Carvalho (2014) propose a new mixed-integer formulation which they solve with a commercial solver. Masson, Ropke, Lehuédé, and Péton (2014b) study a special case of the PDPT, called the PDP with shuttle routes. In this problem the trips between the origins and the destinations can be decomposed into two legs. The first leg visits only origins and ends at some destination, while the second leg is a direct trip (called a shuttle) between two destinations. They propose a branch-and-cut-and-price algorithm to solve the problem. Both the PDPSL and the PDPT are known to be much more difficult to solve in practice than the PDP. This is due to the additional constraints and aspects which need to be taken into account (such as dealing with multiple flows for the same request and guaranteeing that every request is fully satisfied in the PDPSL, or dealing with the synchronization of vehicles in the PDPT) as well as the larger solution space (PDP solutions are valid for the PDPSL and the PDPT, but not vice versa).

A similar problem to the PDPT is the PDP with cross-docks (PDPC). The main difference between a cross-dock and a trans-

[1] https://ec.europa.eu/eurostat/statistics-explained/index.php/Freight_transported_in_containers_-_statistics_on_unitisation Accessed on 8 January 2020.

shipment location is that a cross-dock is the starting and ending vertex of each route, whereas a transshipment location can be visited at any position in the route. Also, cross-docks typically provide more elaborate handling operations (such as labelling, repacking and consolidation) in addition to transshipment. In the PDPC each request is therefore usually required to transit through a cross-dock. Grangier, Gendreau, Lehuédé, and Rousseau (2017) propose a matheuristic algorithm for the PDPC which is based on an ALNS and periodically solving a set partitioning and matching problem with commercial solvers. Maknoon and Laporte (2017) develop an ALNS metaheuristic for the same problem. Petersen and Ropke (2011) address a variant of the PDPC where cross-docking is optional. They present a parallel ALNS metaheuristic for this problem. We refer to Guastaroba, Speranza, and Vigo (2016) for a comprehensive review on intermediate facilities in freight transportation.

The contribution of our article is threefold. First, we present a new mixed-integer formulation for the PDPSLT where the vehicle fleet can be heterogeneous and there is no need to discretize the time component. Second, we propose a branch-and-cut algorithm based on valid inequalities for this formulation. Third, we present a computational study which shows the superior performance of our solution approach compared to a straight branch-and-bound strategy.

The remainder of this article is structured as follows. Section 2 introduces the PDPSLT and presents an arc-based mixed-integer linear programming formulation for it. Section 3 describes the valid inequalities used in the branch-and-cut algorithm which is detailed in Section 4. Section 5 analyzes the computational experiments. Conclusions are reported in Section 6.

2. Problem Description

In this section, we give a formal description of the PDPSLT and present an arc-based mixed-integer linear programming formulation for it.

Let R and L be the sets of requests and locations, respectively. Each request $r \in R$ is associated with an origin location $r^+ \in L$, a destination location $r^- \in L$, and a positive load d_r. Each location $i \in L$ may simultaneously serve as both an origin and a destination of different requests. Thus, $|L|$ may be much smaller than $2|R|$, but it can also be larger since there may exist locations which are neither the origin nor the destination of any request.

Let K be a heterogeneous fleet of vehicles. Each vehicle $k \in K$ has a non-negative capacity q_k, a start depot k^+ at which it must start its tour, and an end depot k^- at which it must end its tour. The locations of the start depot and end depot can coincide for a vehicle. The set of depots is denoted by $N = \bigcup_{k \in K} \{k^+, k^-\}$.

The load of a request may be split. This means that when a vehicle arrives at an origin location it may pick up any portion or the complete load of the request. When the vehicle arrives at the request's destination, all load from this vehicle associated with this request is delivered. As a consequence a customer might be visited by more than one vehicle or more than once by the same vehicle. Furthermore, each part of a request may thus be transported on a different path from the pickup location to the delivery location. We adopt the customary definition of a split: a split occurs when the load of a request is serviced by a larger number of trips than the minimum necessary (see Haddad et al., 2018). For example, when $d_r = 6$ and $q_k = 5$, a split occurs when the request r is delivered through three trips, but not when it is delivered by two.

Furthermore, load may be transshipped from one vehicle to another at specific locations. Load can also be unloaded from a vehicle in these locations, and reloaded to the same vehicle later. Let $T \subseteq L$ be the set of transshipment locations. Transshipment is allowed only at those locations in T. We do not limit the amount of times a request (or a part of it) can be transshipped.

The PDPSLT allows that each location $i \in L$ may be visited more than once by the same vehicle. Multiple visits by the same vehicle may occur when a request is split, when the sum of the demands of all requests at a location is larger than the vehicle capacity, or at a transshipment location. We assume to be given an upper bound n_i on the maximum number of allowed visits by the same vehicle at location i. In order to facilitate the mathematical formulation, we use location-copies. Let V_i be the set of n_i copies of location $i \in L$, and let us denote by $l(v)$ the location associated with a visit v, i.e. $l(v) = i$ if $v \in V_i$. Then, the problem is defined on a directed graph $G = (V, A)$ where the vertex set is $V = N \cup \bigcup_{i \in L} V_i$ and the arc set is $A = \bigcup_{k \in K} A^k$, with $A^k \subset V \times V$ representing the feasible movements of each vehicle $k \in K$.

Each vehicle $k \in K$ and each arc $a \in A^k$ is associated with a travel cost c_a^k and a travel time t_a^k. We assume that the triangle inequality holds for travel costs and travel times. Furthermore, each location $i \in T$ is associated with a cost b_i per unit of transshipped load.

A route for a vehicle corresponds to a path from its start depot to its end depot in the graph G. The path is feasible if the vehicle capacity is never exceeded. In case a transshipment takes place, the vehicle which receives the load must not start its service at the transshipment vertex earlier than the vehicle which gives the load.

The objective of the PDPSLT is to find a set of feasible vehicle routes which serves all requests, such that the sum of travel costs and transshipment costs is minimized.

We use the following mathematical variables to formulate the PDPSLT:

s_v^k start time of service of vehicle $k \in K$ at visit $v \in V$

f_{ar}^k amount of request $r \in R$ transported by vehicle $k \in K$ over arc $a \in A$

$g_{vv'r}^{kk'}$ amount of request $r \in R$ transshipped from vehicle k at visit v to vehicle k' at visit v', $l(v) = l(v')$

x_a^k $\begin{cases} 1 \text{ iff vehicle } k \in K \text{ traverses arc } a \in A \\ 0 \text{ otherwise} \end{cases}$

y_v^k $\begin{cases} 1 \text{ iff vehicle } k \in K \text{ visits vertex } v \in V \\ 0 \text{ otherwise} \end{cases}$

$z_{vv'}^{kk'}$ $\begin{cases} 1 \text{ iff load is transshipped from vehicle } k \text{ at visit } v \\ \quad \text{to vehicle } k' \text{ at visit } v',\ l(v) = l(v') \\ 0 \text{ otherwise.} \end{cases}$

The location-copies allow representing the route of a vehicle k by an elementary path in the graph $G^k = (V, A^k)$, even when the route visits the same location several times. An elementary path can be modeled with the binary variables x_a^k and y_v^k, as typically done in other vehicle routing problems in the literature. Note that when $k = k'$ then $z_{vv'}^{kk'} = 1$ represents a reload to vehicle k at location $l(v)$, while a proper transshipment between different vehicles occurs when $k \neq k'$. Both cases must happen at the same location, which is the reason for having variables g and z when $l(v) = l(v')$ only. It also helps reducing the number of those variables in the formulation.

To abbreviate notation, for any subset $S \subseteq V$, we write $\delta_k^+(S) = \left\{(u, v) \in A^k \mid u \in S, v \notin S\right\}$ and $\delta_k^-(S) = \left\{(u, v) \in A^k \mid u \notin S, v \in S\right\}$. Also, for any subset $F \subseteq A^k$, we write $x^k(F) = \sum_{a \in F} x_a^k$ and $f_r^k(F) = \sum_{a \in F} f_{ar}^k$.

Then, the PDPSLT can be formulated as the following arc-based mixed-integer linear program:

$$\min \sum_{k \in K} \sum_{a \in A^k} c_a^k x_a^k + \sum_{r \in R} \sum_{i \in T} \sum_{v, v' \in V_i} \sum_{k, k' \in K} b_i g_{vv'r}^{kk'} \tag{1}$$

subject to

$$x^k(\delta_k^+(v)) = y_v^k \qquad \forall k \in K, \forall v \in V \setminus \{k^-\} \qquad (2)$$

$$x^k(\delta_k^-(v)) = y_v^k \qquad \forall k \in K, \forall v \in V \setminus \{k^+\} \qquad (3)$$

$$y_v^k = 1 \qquad \forall k \in K, \forall v \in \{k^+, k^-\} \qquad (4)$$

$$\sum_{k \in K} f_r^k(\delta_k^+(V_{r^+})) = \sum_{k \in K} f_r^k(\delta_k^-(V_{r^-})) = d_r \qquad \forall r \in R \qquad (5)$$

$$\sum_{k \in K} f_r^k(\delta_k^-(V_{r^+})) = \sum_{k \in K} f_r^k(\delta_k^+(V_{r^-})) = 0 \qquad \forall r \in R \qquad (6)$$

$$f_r^k(\delta_k^-(v)) + \sum_{k' \in K} \sum_{v' \in V_i} (g_{v'vr}^{k'k} - g_{vv'r}^{kk'}) = f_r^k(\delta_k^+(v)) \qquad \forall r \in R, \forall i \in T \setminus \{r^+, r^-\}, \forall v \in V_i, \forall k \in K \qquad (7)$$

$$f_r^k(\delta_k^+(v)) = f_r^k(\delta_k^-(v)) \qquad \forall r \in R, \forall i \in L \setminus (\{r^+, r^-\} \cup T), \forall v \in V_i, \forall k \in K \qquad (8)$$

$$s_u^k + t_{(u,v)}^k - M(1 - x_{(u,v)}^k) \le s_v^k \qquad \forall k \in K, \forall (u,v) \in A^k \qquad (9)$$

$$\sum_{r \in R} f_{ar}^k \le q_k x_a^k \qquad \forall k \in K, \forall a \in A^k \qquad (10)$$

$$s_v^k - M(1 - z_{vv'}^{kk'}) \le s_{v'}^{k'} \qquad \forall i \in T, \forall v, v' \in V_i, \forall k, k' \in K \qquad (11)$$

$$g_{vv'r}^{kk'} \le d_r z_{vv'}^{kk'} \qquad \forall r \in R, \forall i \in T, \forall v, v' \in V_i, \forall k, k' \in K \qquad (12)$$

$$z_{vv'}^{kk'} \le y_v^k \quad ; \quad z_{vv'}^{kk'} \le y_{v'}^{k'} \qquad \forall i \in T, \quad \forall v, v' \in V_i, \forall k, k' \in K \qquad (13)$$

$$z_{vv'}^{kk'} = 0 \qquad \forall i \notin T, \forall v, v' \in V_i, \forall k, k' \in K \qquad (14)$$

$$x_a^k, y_v^k, z_{vv'}^{kk'} \in \{0,1\} \quad ; \quad f_{ar}^k, g_{vv'r}^{kk'}, s_v^k \ge 0 \qquad \forall r \in R, \forall v, v' \in V, \forall k, k' \in K, \forall a \in A^k. \qquad (15)$$

The objective function (1) aims at minimizing the total cost. Constraints (2) and (3) guarantee the flow conservation of vehicles. Constraints (4) make sure that every vehicle starts and ends its path at the depot. Constraints (5) guarantee that every request is fully serviced. Constraints (6) forbid that load of a request enters (leaves) its own origin (destination). Constraints (7) regulate the flow conservation of load at every transshipment location, while constraints (8) forbid transshipment at pure customer locations. Time consistency is ensured by constraints (9). This inequality needs a big value M which can be computed individually for each combination of u, v and k as $M_{uv}^k = s_u^{kMAX} + t_{(u,v)}^k - s_v^{kMIN}$, where s_u^{kMAX} denotes the latest feasible arrival time of vehicle k at vertex u and s_v^{kMIN} denotes the earliest feasible arrival time of vehicle k at vertex v. Constraints (10) ensure that the capacity of a vehicle is never exceeded. Constraints (11) ensure time synchronization at transshipment. This inequality also needs a big value M. Like with constraints (9), it can be computed individually for each combination of v, k and k' as $M_v^{kk'} = s_v^{kMAX} - s_v^{k'MIN}$. Constraints (12) guarantee that the load of a request transshipped from one vehicle to another does not exceed the total demand of the request. The linking constraints (13) allow a transshipment at a vertex from or to a vehicle only if the vehicle visits that vertex. Constraints (14) forbid transshipment at all locations $i \notin T$. Constraints (15) set the domain of the decision variables. Given a base graph with m vertices (i.e., locations) this formulation has $\mathcal{O}(n_i \cdot |K| \cdot |R| \cdot m^2)$ variables and a polynomial number of constraints. For comparison, the formulation in Kerivin et al. (2008) for a particular case of the PDPSLT has $\mathcal{O}(\mathcal{T} \cdot m^2)$ variables (which is exponential in the size of the instances because $\mathcal{T}$ is an input parameter) and an exponential number of constraints.

Fig. 1 illustrates a small problem instance of the PDPSLT as formulated above. The instance consists of five locations (1, 2, 3, 4, T), six requests (A, B, C, D, E, F) and two depots. Transshipment is only allowed at location T. The origins of requests A, B and C are at location 1 and the origins of the other three requests are at location 2. Requests A, C and E must be delivered to their destinations at location 3, while B, D and F must be delivered to their destinations at location 4. The numbers in brackets next to the requests are the demand. One depot is located at location 1, the other at location 2. Six vehicles with capacity 10 are available in total; three vehicles at each depot. To keep it simple, the transshipment cost is equal to zero. The numbers next to the edges are the distances.

Fig. 2 illustrates four optimal solutions of the problem instance in Fig. 1 depending on whether split loads and/or transshipments are allowed in the problem description (i.e., optimal solutions for the PDP, PDPT, PDPSL and PDPSLT). The solutions show that considering split loads and transshipment simultaneously may result in even lower costs than considering only one of these two extensions. In the case where neither split loads nor transshipments are allowed (PDP, Fig. 2a) the optimal solution has a cost of 193.14 and uses two vehicles. The first vehicle delivers A, D and B, and the second vehicle delivers C, E and F. For the case where transshipment is allowed (PDPT, Fig. 2b), the optimal solution does not change. That is, allowing transshipment does not add any possibilities to reduce costs. On the other hand, when split loads are allowed (PDPSL, Fig. 2c), the cost can be reduced by approximately 20% to 153.14. The optimal solution in this case splits the loads of requests C and D and uses two vehicles as well. Vehicle 1 starts by picking up the complete load of A as well as two units of C, which it then delivers to their destinations. Afterwards, it picks up six units of D and the complete load of E. Then, the vehicle delivers E before it delivers the six units of D and travels back to the depot. Vehicle 2 starts by picking up two units of D and F completely. It delivers them to their destinations and travels further on to pick up B completely and six units of C. Vehicle 2 first delivers B, then the six units of C and then returns to the depot. When both split loads and transshipments are allowed (PDPSLT, Fig. 2d) the costs are reduced further by approximately 10% to 136.57. In addition to C and D being split, B and E are transshipped in the optimal solution. Like in the previous case (Fig. 2c), vehicle 1 starts by delivering A completely as well as two units of C. The vehicle then picks up six units of D and the complete load of E. It then travels to the transshipment location where it gives E (four units) to, and receives B (four units) from, vehicle 2. Afterwards the vehicle delivers B and the six units of D, and then returns to the depot. Vehicle 2 starts by delivering two units of D and the complete load of F. It then picks up the full load of B and six units of C. Afterwards it travels to the transshipment location where it receives E from, and gives B to, vehicle 1. After the transshipments took place, vehicle 2 delivers both requests and returns to the depot.

2.1. Comments on the Number of Location-Copies

In order to work on a graph where each vertex is visited at most once, we need to be given an explicit number of copies n_i for each location $i \in L$, i.e., $n_i = |V_i|$. The value of n_i can be set arbitrarily large such that it does not impose any limitation on the construction of vehicle routes. However, finding the right (i.e., minimum) value of n_i for each location such that the solution space contains the optimal solution for this unlimited case is not trivial. To the best of our knowledge no algorithm exists to calculate this value. Obviously, if there is an inherent upper bound to this value in a specific practical application (e.g., customers do not want to be visited more than a certain number of times) then n_i can safely be set to this upper bound without losing optimality.

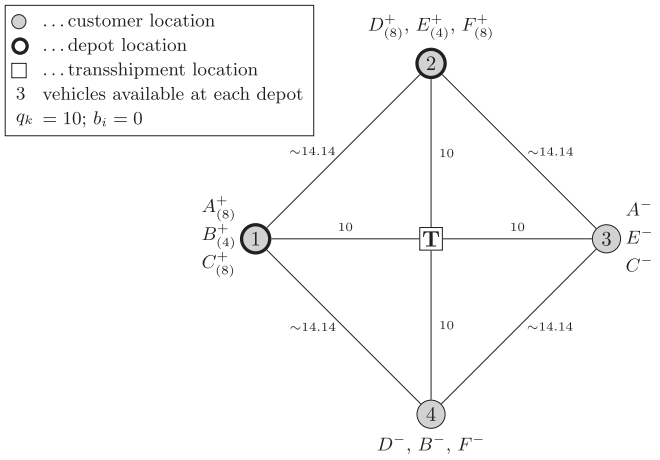


Fig. 1. Small instance with five locations (1, 2, 3, 4, T) and six requests (*A*, *B*, *C*, *D*, *E*, *F*). Transshipment is only allowed at location T. The numbers in brackets next to the requests are the demand. The numbers next to the edges are the distances.

2.2. Time Restrictions

In a variety of real-world applications of the PDP (and its variants) temporal aspects play an important role. For instance, labor regulations might impose a maximum duration on vehicle routes, or customers might be available for service only during certain periods of time (requiring the consideration of time windows at customer locations). It is not difficult to extend the above stated formulation such that temporal requirements are additionally taken into account. While these requirements make the PDPSLT more realistic, they also make it much more difficult to solve, though.

To additionally consider a maximum route duration τ, the following constraints could be added:

$$s^k_{k^-} \leq \tau \qquad \forall k \in K. \tag{16}$$

The formulation (1)–(15) could also be extended to consider time-window restrictions as follows. Let $[a_i, b_i]$ be the time window associated with location i. Then, compliance with the time interval can be ensured with the following constraints:

$$a_i \leq s^k_v \leq b_i \qquad \forall i \in L, \forall v \in V_i, \forall k \in K. \tag{17}$$

Formulation (1)–(15) does not consider service times. They are unnecessary because no time restrictions are present in our definition of the PDPSLT. However, if time restrictions were present in the problem definition, accounting for service times can be important (e.g., in the context of multimodal transportation where loading operations can take up a considerable amount of time). For such cases, a service time ω^k_v needs to be associated with each vehicle $k \in K$ and each vertex $v \in V$. Then, constraints (9) and (11) need to be replaced with

$$s^k_u + \omega^k_u + t^k_{(u,v)} - M(1 - x^k_{(u,v)}) \leq s^k_v \qquad \forall k \in K, \forall (u,v) \in A^k \tag{18}$$

$$s^k_v + \omega^k_v - M(1 - z^{kk'}_{vv'}) \leq s^{k'}_{v'} \qquad \forall i \in T, \forall v, v' \in V_i, \forall k, k' \in K, \tag{19}$$

respectively, where M represents a big value.

For simplicity in the exposition of our research results, we do not include these additional features in the formulation.

3. Valid Inequalities

This section describes several families of valid inequalities for the PDPSLT. These inequalities are redundant for formulation (1)–(15) but can be used to strengthen its Linear Programming relaxation. We use the following additional notation to describe the valid inequalities. Let $x^k(S) = \sum_{i,j \in S} x^k_{(i,j)}$ for any vertex set $S \subseteq V$ and vehicle $k \in K$. For notational convenience, let $x(F) = \sum_{k \in K} x^k(F)$, and similarly $\delta^+(S) = \bigcup_{k \in K} \delta^+_k(S)$ and $\delta^-(S) = \bigcup_{k \in K} \delta^-_k(S)$. Furthermore, let $Q = \max\{q_k \mid k \in K\}$ denote the largest capacity of all vehicles and let $R^+_i = \{r \in R \mid i = r^+\}$ and $R^-_i = \{r \in R \mid i = r^-\}$ be the set of requests whose origin, respectively destination, is at location $i \in L$. Finally, if S' and S'' are subsets of V, we define $x(S' : S'') = \sum_{k \in K} \sum_{i \in S'} \sum_{j \in S''} x^k_{(i,j)}$ as the total flow from vertices in S' to vertices in S''.

3.1. Subtour Elimination Constraints

For each vehicle $k \in K$, any subset $S \subset V$ such that $k^- \notin S$ and any vertex $i \in S$, the inequality

$$x^k(\delta^+_k(S)) \geq y^k_i \tag{20}$$

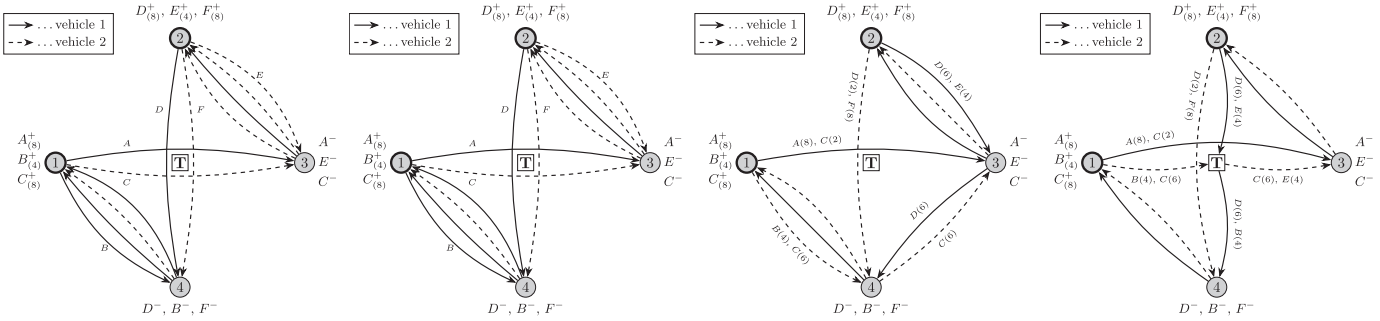
is valid and prohibits subtours.

3.2. Infeasible Path Constraints

Infeasible path constraints are standard inequalities in the Vehicle Routing literature to avoid invalid paths in a solution. For the PDPSLT an invalid path may appear when violating the precedence relationship between the origin and destination of a request. We next derive these constraints.

Let $\mathcal{P} = (v_1, \ldots, v_m)$ be an infeasible path. Consider a vehicle $k \in K$. Ascheuer, Fischetti, and Grötschel (2000) propose the following inequalities (named tournament constraints) for the Asymmetric Traveling Salesman Problem with Time Windows (ATSPTW) to forbid infeasible paths:

$$\sum_{1 \leq i < j \leq m} x^k_{(v_i,v_j)} \leq m - 2.$$



(a) PDP (cost: 193.14) **(b)** PDPT (cost: 193.14) **(c)** PDPSL (cost: 153.14) **(d)** PDPSLT (cost: 136.57)

Fig. 2. Optimal solutions of the instance in Fig. 1 depending on whether split loads and/or transshipments are allowed. The numbers in brackets next to the requests on the edges are the transported loads of the respective requests.

Due to the presence of both the possibility of transshipments and the possibility of two or more requests sharing the same location, the feasibility of a path $\mathcal{P}$ in the PDPSLT does not solely depend on the sequence of vertices in the path. It additionally depends on which requests are serviced or transshipped along the path. Let us give an example with two requests. Assume $\mathcal{P} = (0, 1, 2, 0)$, where 0 is the depot, 1 is the origin of request A and the destination of request B, and 2 is the destination of request A and the origin of request B. Clearly, $\mathcal{P}$ is valid if it only serves request A, but invalid when it serves request B. Therefore, to make the above inequality valid for the PDPSLT we need to include the information about which requests are transshipped and/or serviced along the path in the constraints. To this end, one can introduce a new binary variable e^k_{vr} assuming value 1 if, and only if, request $r \in R$ is serviced by vehicle $k \in K$ at vertex $v \in V$, and value 0 otherwise. We must add then the inequalities

$$e^k_{vr} \leq y^k_v \qquad \forall v \in V, \forall r \in R, \forall k \in K \tag{21}$$

$$f^k_r\left(\delta^+(v)\right) - f^k_r\left(\delta^-(v)\right) \leq d_r e^k_{vr} \qquad \forall v \in V, \forall r \in R, \forall k \in K \tag{22}$$

$$f^k_r\left(\delta^-(v)\right) - f^k_r\left(\delta^+(v)\right) \leq d_r e^k_{vr} \qquad \forall v \in V, \forall r \in R, \forall k \in K \tag{23}$$

$$e^k_{vr} \in \{0, 1\} \qquad \forall r \in R, \forall v \in V, \forall k \in K. \tag{24}$$

Then, the inequality

$$\sum_{1 \leq i < j \leq m} x^k_{(v_i, v_j)} \leq m - e^k_{v_p r} - e^k_{v_q r} \tag{25}$$

is valid for every vehicle $k \in K$, every request $r \in R$ and every path $\mathcal{P}$ where $v_p \in V_{r^-}$, $v_q \in V_{r^+}$ and $p < q$, i.e. a visit to r^- precedes a visit to r^+ in the path $\mathcal{P}$.

3.3. Capacity Constraints

Let $\bar{d}_i = \sum_{r \in R^+_i} d_r - \sum_{r \in R^-_i} d_r$ be the supply at each location $i \in L$. Furthermore, for any $\mathcal{S} \subseteq L$ let $S = \{v \in V_i \mid i \in \mathcal{S}\}$, and denote the minimum number of vehicles needed to service all vertices in S by $W(S)$. The following inequality is then valid:

$$x(\delta^-(S)) \geq W(S). \tag{26}$$

Computing $W(S)$ is difficult. However, a lower approximation is given by $\lceil d(S)/Q \rceil$, where $d(S) = \sum_{i \in \mathcal{S}} \bar{d}_i$. The resulting constraint is called Rounded Capacity Inequality (see for example Naddef & Rinaldi (2002)).

Due to the presence of both positive and negative d_r values when calculating $\bar{d}_i$, these inequalities are not likely to be very strong. Because of that, we consider either only the pickups or only the deliveries in S, as done in Cordeau (2006). In our case, we further restrict S such that for all $r \in R$ either $V_{r^-} \cap S = \emptyset$ or $V_{r^+} \cap S = \emptyset$, and replace the calculation of $\bar{d}_i$ with either $\bar{d}_i = \sum_{r \in R^+_i} d_r$ or $\bar{d}_i = \sum_{r \in R^-_i} d_r$, accordingly.

As also observed in Cordeau (2006), in our context the quantity $\lceil d(S)/Q \rceil$ estimates the required number of vehicle *passages* in S rather than the number of vehicles. In fact, the same vehicle may be able to visit all vertices in S by leaving and entering the set several times, even if $d(S) > Q$.

3.4. Strengthened Capacity Constraints

Ropke, Cordeau, and Laporte (2007) propose a strengthened version of the rounded capacity constraints for the PDP with time windows which is also valid for the PDPSLT. The idea behind these new constraints is to consider requests such that the pickup location r^+ is visited before entering set S, while the delivery location r^- is visited after leaving S. In this case, the required number of vehicle passages in set S increases in order to accommodate the demand of all such requests. While the actual inequality proposed and used by Ropke et al. (2007) is not valid for the PDPSLT, its original form is valid for the PDPSLT.

Let $\pi(S) = \{v \in V_{r^+} \mid V_{r^-} \cap S \neq \emptyset, \forall r \in R\}$ be the set of all pickup vertices for which the corresponding delivery vertex is in set S. Furthermore, let $\mathcal{S}' \subset L$ and $\mathcal{S}'' \subset L$ be two disjoint sets, and set $S' = \{v \in V_i \mid i \in \mathcal{S}'\}$ and $S'' = \{v \in V_i \mid i \in \mathcal{S}''\}$ such that $d(S') > 0$. Additionally, let $U = \pi(S'') \setminus (S' \cup S'')$. Due to the fact that $d(S') > 0$ and $d(U) \geq 0$, $x(\delta^+(S')) \geq \left\lceil d(S')/Q \right\rceil$ and $x(\delta^-(S'')) \geq \lceil d(U)/Q \rceil$. Therefore, taking care not to double count the arcs from set S' to set S'', the following inequality is valid for the PDPSLT:

$$x(\delta^+(S')) + x(\delta^-(S'')) - x(S' : S'') \geq \left\lceil \frac{d(S') + d(U)}{Q} \right\rceil. \tag{27}$$

It is not difficult to see that inequality (27) is a strengthening of the rounded capacity inequality (26). From inequality (26), equation $x(\delta^+(S')) = x(\delta^-(S'))$ and inequality $x(\delta^-(S'')) - x(S' : S'') \geq 0$, one obtains

$$x(\delta^+(S')) + x(\delta^-(S'')) - x(S' : S'') \geq \left\lceil \frac{d(S')}{Q} \right\rceil,$$

which is weaker than (27) if $d(U) > 0$.

3.5. Predecessor/Successor Constraints

Any pickup-and-delivery action imposes precedence relations between visits. Balas, Fischetti, and Pulleyblank (1995) introduced families of inequalities to exploit precedence relations in a vehicle routing problem. We have adapted some of these inequalities to the PDPSLT as follows. Let $P = (V, D)$ denote a precedence graph which is defined on the same node set V as G. An arc $(u, v) \in D$, $u, v \notin N$, represents a precedence relationship (also denoted as $u \prec v$), i.e. u has to precede v in every feasible tour.

Let $\pi(j) = \{i \in V \mid (i, j) \in D\}$ be the set of predecessors of j, and $\sigma(i) = \{j \in V \mid (i, j) \in D\}$ be the set of successors of i. For any $i \in V \setminus N$ such that $\pi(i) \neq \emptyset$, $l(j) \notin T$, $|R^+_{l(j)}| = 1$ and $|R^-_{l(j)}| = 0$ for all $j \in \pi(i)$, any $S \subset V$ such that $i \in S$, and using $\bar{S} = V \setminus S$, the inequality

$$x\left(S \setminus \pi(i) : \bar{S} \setminus \pi(i)\right) \geq 1 \tag{28}$$

is valid for the PDPSLT and called a *weak π-constraint*.

Similarly, for any $i \in V \setminus N$ such that $\sigma(i) \neq \emptyset$, $l(j) \notin T$, $|R^+_{l(j)}| = 0$ and $|R^-_{l(j)}| = 1$ for all $j \in \sigma(i)$, any $S \subset V$ such that $i \in S$, and again $\bar{S} = V \setminus S$, the inequality

$$x\left(S \setminus \sigma(j) : \bar{S} \setminus \sigma(j)\right) \geq 1 \tag{29}$$

is also valid for the PDPSLT and called a *weak σ-constraint*.

Furthermore, for any $i, j \in V$ where $i \prec j$ and $W = N \cup \pi(i) \cup \sigma(j)$

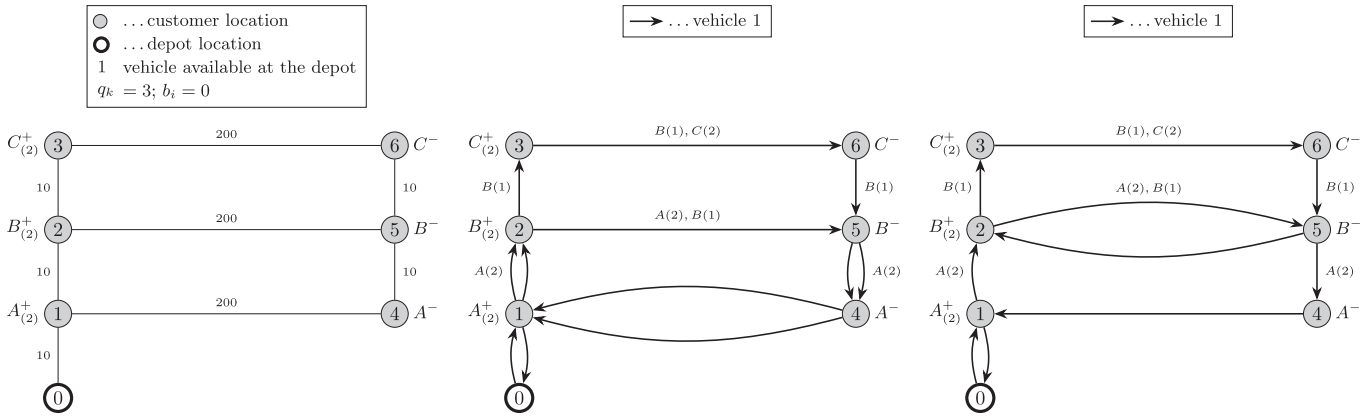
$$x\left(S \setminus W : \bar{S} \setminus W\right) \geq 1 \tag{30}$$

is also valid for the PDPSLT and called a *weak $\pi - \sigma$-constraint*.

4. Branch-and-Cut Algorithm

In this section, we describe a branch-and-cut algorithm to solve the PDPSLT. We focus on the separation heuristic procedures used to identify violated valid inequalities and refer to Wolsey (1998) and Mitchell (2011) for a thorough description of the technique.

The algorithm starts by solving the linear relaxation of the model (1)–(15) and (21)–(23). If the solution is integer on the e, x, y, and z variables, an optimal PDPSLT solution has been found and the algorithm stops. Otherwise, a branch-and-bound tree is



(a) Instance **(b)** PDPSLT (cost: 880) from Kerivin et al. (2008) **(c)** PDPSLT (cost: 860) from our approach

Fig. 3. Small instance from Kerivin et al. (2008) and two PDPSLT solutions.

Table 1
Contribution of valid inequalities (average values over 36 instances).

	Indicator	Compact	SEC	IPC	CC	SCC	PRC	B&C
$n_i = 1$	LB	87.79	87.20	86.87	87.89	87.33	87.69	91.34
	Time	275.41	328.28	459.20	411.31	430.01	477.36	156.01
	# cuts	0	3423	182	72	314	1602	2131
	# solved	30	30	27	28	27	27	34
	Gap	3.56	6.04	7.17	5.28	7.18	8.48	3.15
$n_i = 2$	LB	82.51	82.33	82.53	84.24	84.57	84.68	88.98
	Time	1350.82	1387.56	1328.06	1280.93	1236.52	1215.16	1121.56
	# cuts	0	20632	258	74	402	6702	12248
	# solved	3	2	4	6	7	7	9
	Gap	9.37	13.01	11.41	10.43	9.74	11.96	8.87
$n_i = 3$	LB	76.57	76.55	76.50	79.56	81.42	81.64	87.36
	Time	1280.97	1440.00	1422.75	1395.77	1381.75	1440.00	1440.00
	# cuts	0	56186	187	64	343	19349	16484
	# solved	6	0	1	2	3	0	0
	Gap	13.57	19.22	16.28	13.83	13.20	16.13	11.12

initiated to be explored. We generate violated inequalities at each node of this tree using the separation heuristics described in the following.

Let us introduce some additional notation useful to describe the cut separation procedures. Given a solution $[e^*, f^*, g^*, s^*, x^*, y^*, z^*]$ of a linear relaxation of the mathematical formulation, let $G^{k*} = (V^{k*}, A^{k*})$ be an auxiliary graph for vehicle $k \in K$ where $V^{k*} = \{v \in V \mid y_i^{k*} > 0\}$ is the vertex set, $A^{k*} = \{a \in A^k \mid x_a^{k*} > 0\}$ is the arc set, and x_a^{k*} is the capacity of each arc $a \in A^{k*}$.

4.1. Subtour Elimination Constraints

We perform the following three steps for each vehicle $k \in K$ in order to identify subtours. First, we check if the inequalities $x^k_{(i,j)} + x^k_{(j,i)} \leq 1$ are violated. Second, if no violation was found, we check whether graph G^{k*} is connected using depth first search. In case it is not connected, each connected component induces a violated subtour elimination constraint (20). Third, if still no violation was found, we compute the cut with minimum capacity in G^{k*} in order to identify a violated inequality (20). We use the minimum s-t cut algorithm of the Concorde callable library (see Applegate, Bixby, Chvátal, & Cook (2003)) to compute this cut.

4.2. Infeasible Path Constraints

We use a heuristic method similar to that of Ropke et al. (2007) to identify infeasible paths violating inequalities (25). For each vehicle $k \in K$ the method considers the vehicle's start depot k^+ as start vertex from which it constructs a tree of paths which consist only of arcs from set A^{k*}. Each path is extended as long as the total flow on the arcs in path $\mathcal{P}$ is strictly greater than $|\mathcal{P}| - 2$ and the path has not reached the vehicle's end depot k^-. Whenever an infeasible path is detected, the corresponding infeasible path constraint (25) is generated.

4.3. Capacity Constraints

We use the tabu search heuristic described by Cordeau (2006) to identify violated capacity constraints (26). The heuristic procedure starts with a random subset $\mathcal{S} \subseteq L$ where $d(S)$ is calculated either only for the pickups or only for the deliveries. At each iteration, a location is either added or removed from set $\mathcal{S}$ so as to minimize the value of $x^*(\delta^-(S))$ while satisfying $d(S) > Q$. During a single execution of this heuristic multiple violated capacity constraints may be identified. In case no violated constraints were identified, we re-initialize set $\mathcal{S}$ and re-calculate $d(S)$ with the method not used in the previous execution (i.e., calculate $d(S)$ either only for the pickups or only for the deliveries) and run the heuristic again. The procedure runs for 10 iterations in each execution.

4.4. Strengthened Capacity Constraints

To identify violated strengthened capacity constraints (27) we use an adapted version of the iterative construction heuristic proposed by Ropke et al. (2007). It starts from a set $\mathcal{S}'$ containing a

Table 2
Computing times in minutes and percentage improvement when $n_i > 1$.

Name	B&C	B&C-WTC	Name	B&C		B&C-WTC		Name	B&C		B&C-WTC	
				Time	Improv.	Time	Improv.		Time	Improv.	Time	Improv.
5-2-1-1-∗	0.73	0.78	5-2-1-2-∗	34.70	0.77	32.66	0.77	5-2-1-3-∗	828.75	0.07	983.86	−0.13
5-2-2-1-∗	4.52	2.50	5-2-2-2-∗	316.20	0.38	306.86	0.77	5-2-2-3-∗	1185.72	−0.71	1155.57	−11.02
5-2-3-1-∗	4.21	4.07	5-2-3-2-∗	458.18	0.77	327.85	0.77	5-2-3-3-∗	1338.66	−2.14	1440.00	−0.20
5-3-1-1-∗	4.31	4.13	5-3-1-2-∗	589.49	0.23	471.51	0.21	5-3-1-3-∗	1154.91	−2.02	1152.37	−3.45
5-3-2-1-∗	4.42	5.97	5-3-2-2-∗	920.33	0.19	674.28	0.21	5-3-2-3-∗	1152.36	−3.78	1152.61	−4.22
5-3-3-1-∗	8.86	9.71	5-3-3-2-∗	1154.14	0.07	929.54	−0.03	5-3-3-3-∗	1152.78	−8.10	1153.65	−3.03
5-∗-∗-1-∗	4.51	4.52	5-∗-∗-2-∗	578.84	0.40	457.12	0.45	5-∗-∗-3-∗	1135.53	−2.78	1173.01	−3.68
6-2-1-1-∗	6.67	7.22	6-2-1-2-∗	484.77	2.28	344.85	2.27	6-2-1-3-∗	1175.64	0.52	1152.80	−17.68
6-2-2-1-∗	9.43	8.31	6-2-2-2-∗	645.04	2.12	746.39	2.43	6-2-2-3-∗	1153.35	−9.85	1152.76	−24.43
6-2-3-1-∗	19.70	17.83	6-2-3-2-∗	736.77	2.66	807.98	2.47	6-2-3-3-∗	1153.91	−16.01	1153.94	−21.73
6-3-1-1-∗	7.17	8.78	6-3-1-2-∗	143.21	2.29	290.78	2.29	6-3-1-3-∗	1440.00	−14.80	1440.00	−4.75
6-3-2-1-∗	11.75	11.62	6-3-2-2-∗	429.21	0.73	292.22	2.29	6-3-2-3-∗	1440.00	−10.54	1440.00	−1.03
6-3-3-1-∗	11.50	11.01	6-3-3-2-∗	838.84	2.29	888.68	2.23	6-3-3-3-∗	1440.00	−39.85	1440.00	−15.05
6-∗-∗-1-∗	11.04	10.80	6-∗-∗-2-∗	546.31	2.06	561.82	2.33	6-∗-∗-3-∗	1300.48	−15.08	1296.58	−14.11
7-2-1-1-∗	9.35	9.42	7-2-1-2-∗	875.27	0.35	966.15	0.38	7-2-1-3-∗	1440.00	−1.85	1440.00	−64.34
7-2-2-1-∗	8.74	10.36	7-2-2-2-∗	1147.93	0.39	911.82	−1.55	7-2-2-3-∗	1440.00	−4.53	1440.00	−6.44
7-2-3-1-∗	12.82	11.92	7-2-3-2-∗	895.58	0.02	1016.40	−0.89	7-2-3-3-∗	1440.00	−29.06	1440.00	−23.30
7-3-1-1-∗	6.57	5.18	7-3-1-2-∗	1179.47	−0.05	1281.43	−1.01	7-3-1-3-∗	1440.00	−21.72	1440.00	−51.79
7-3-2-1-∗	8.48	9.23	7-3-2-2-∗	1440.00	−6.34	1440.00	−4.12	7-3-2-3-∗	1440.00	−15.47	1440.00	−13.22
7-3-3-1-∗	11.72	13.46	7-3-3-2-∗	1363.52	−8.28	1440.00	−4.00	7-3-3-3-∗	1440.00	−1.55	1440.00	−41.26
7-∗-∗-1-∗	9.61	9.93	7-∗-∗-2-∗	1150.29	−2.32	1175.97	−1.86	7-∗-∗-3-∗	1440.00	−12.36	1440.00	−33.39
8-2-1-1-∗	17.32	16.17	8-2-1-2-∗	1369.42	−3.45	1440.00	−4.77	8-2-1-3-∗	1440.00	−54.24	1440.00	−45.65
8-2-2-1-∗	28.79	26.49	8-2-2-2-∗	1310.00	−10.50	1440.00	−16.19	8-2-2-3-∗	1440.00	−18.71	1440.00	−30.01
8-2-3-1-∗	58.79	49.04	8-2-3-2-∗	1209.27	−2.37	1324.55	−17.09	8-2-3-3-∗	1440.00	–	1440.00	–
8-3-1-1-∗	17.52	20.20	8-3-1-2-∗	1440.00	−7.03	1440.00	−8.71	8-3-1-3-∗	1440.00	−104.90	1440.00	−2.48
8-3-2-1-∗	37.92	38.22	8-3-2-2-∗	1440.00	−10.97	1440.00	−5.97	8-3-2-3-∗	1440.00	−27.27	1440.00	−2.60
8-3-3-1-∗	87.48	64.78	8-3-3-2-∗	1440.00	−10.74	1440.00	−18.83	8-3-3-3-∗	1440.00	−34.32	1440.00	−46.86
8-∗-∗-1-∗	41.30	35.82	8-∗-∗-2-∗	1368.12	−7.51	1420.76	−11.93	8-∗-∗-3-∗	1440.00	−47.89	1440.00	−25.52
9-2-1-1-∗	92.88	85.74	9-2-1-2-∗	1440.00	−2.67	1440.00	−19.34	9-2-1-3-∗	1440.00	–	1440.00	−9.06
9-2-2-1-∗	303.93	253.99	9-2-2-2-∗	1440.00	−7.57	1440.00	−21.55	9-2-2-3-∗	1440.00	−68.76	1440.00	–
9-2-3-1-∗	568.37	497.84	9-2-3-2-∗	1440.00	–	1440.00	−61.43	9-2-3-3-∗	1440.00	–	1440.00	–
9-3-1-1-∗	13.33	14.59	9-3-1-2-∗	1440.00	−25.18	1440.00	−11.84	9-3-1-3-∗	1440.00	–	1440.00	−24.65
9-3-2-1-∗	17.55	18.30	9-3-2-2-∗	1440.00	−6.85	1440.00	−18.50	9-3-2-3-∗	1440.00	–	1440.00	–
9-3-3-1-∗	28.02	31.18	9-3-3-2-∗	1440.00	−8.10	1440.00	−19.81	9-3-3-3-∗	1440.00	–	1440.00	–
9-∗-∗-1-∗	170.68	150.27	9-∗-∗-2-∗	1440.00	−10.07	1440.00	−25.41	9-∗-∗-3-∗	1440.00	−68.76	1440.00	−16.85
10-2-1-1-∗	703.26	577.29	10-2-1-2-∗	1440.00	−4.29	1440.00	−57.24	10-2-1-3-∗	1440.00	–	1440.00	–
10-2-2-1-∗	822.66	763.92	10-2-2-2-∗	1440.00	–	1440.00	–	10-2-2-3-∗	1440.00	–	1440.00	–
10-2-3-1-∗	976.10	940.94	10-2-3-2-∗	1440.00	−17.35	1440.00	−11.54	10-2-3-3-∗	1440.00	–	1440.00	–
10-3-1-1-∗	585.84	584.74	10-3-1-2-∗	1440.00	−47.56	1440.00	−1.97	10-3-1-3-∗	1440.00	–	1440.00	–
10-3-2-1-∗	600.89	597.08	10-3-2-2-∗	1440.00	−0.57	1440.00	−81.22	10-3-2-3-∗	1440.00	–	1440.00	–
10-3-3-1-∗	618.03	616.60	10-3-3-2-∗	1440.00	−74.77	1440.00	−37.01	10-3-3-3-∗*	1440.00	–	1440.00	–
Average	159.16	148.57	Average	1087.26	−7.03	1082.61	−11.64	Average	1366.00	−20.40	1371.60	−18.74

single location (randomly chosen from the set $\{i \in L \mid \bar{d}_i > 0\}$) and adds one location to $\mathcal{S}'$ in each iteration. Before adding a location, the method determines both the best location l_p and best location l_d to add to the set as follows:

$$l_p \in argmin_{i \in \{j \in L \mid \bar{d}_i > 0\} \setminus \mathcal{S}'} \left\{ x^*(V_i : V \setminus S') - x^*(S' : V_i) \right\}$$

$$l_d \in argmin_{i \in \{j \in L \mid \bar{d}_i < 0\} \setminus \mathcal{S}'} \left\{ x^*(V_i : V \setminus S') - x^*(S' : V_i) \right\}.$$

Preferably, a location l_p will be added to $\mathcal{S}'$ in order to increase $d(S')$ on the right-hand side of (27). Location l_d is only added to set $\mathcal{S}'$ if $(x^*(V_{l_p} : V \setminus S') - x^*(S' : V_{l_p})) > (x^*(V_{l_d} : V \setminus S') - x^*(S' : V_{l_d}))$, $d(S' \cup \{V_{l_d}\}) > 0$ and $x^*(V_{l_d} : V \setminus S') - x^*(S' : V_{l_d}) \leq 0$ or $x^*(V_{l_p} : V \setminus S') - x^*(S' : V_{l_p}) \geq 1$. Whenever a location is added to set $\mathcal{S}'$, the set $\mathcal{S}''$ is reconstructed by using a similar method where the roles of l_p and l_d are interchanged.

At the root node of the enumeration tree, we use a modified version of this heuristic. That is, we restart the heuristic several times from each location $i \in L \mid \bar{d}_i > 0$ and we add a random noise $\varepsilon \sim U[0, 0.5]$ to the evaluation of $\left\{x^*\left(V_i : V \setminus S'\right) - x^*\left(S' : V_i\right)\right\}$.

4.5. Predecessor/Successor Constraints

We adapt the exact separation procedures described in Balas et al. (1995) to separate weak π-, σ-, π–σ-constraints (28), (29) and (30). Let N^+ and N^- be the set of start depots and the set of end depots, respectively. The separation algorithm for weak π-constraints applies the following procedure for each request $r \in R$ whose origin location is pure, i.e. $r^+ \notin T$ and no other request has its origin or destination location at r^+. First we construct an auxiliary graph $\tilde{G} = (\tilde{V}, \tilde{A})$ from G^* by deleting all nodes $i \in V_{r^+}$. Then we try to send one unit of load from a node $s \in V_{r^-}$ to an end depot $t \in N^-$. If we do not succeed, the minimum capacity cut in $\tilde{G}$ separating s from t defines a violated weak π-constraint (28). We again use the minimum s-t cut algorithm of the Concorde callable library to compute this cut. In order to have all location-copies of r^- on one side of the cut and all end depots on the other side, we add all arcs (s, j), where $j \in V_{r^-} \setminus \{s\}$, and all arcs (j, t), where $j \in N^- \setminus \{t\}$, to $\tilde{G}$ and set their capacity to 1 before sending the unit of load, respectively calculating the cut.

The separation of weak σ-constraints follows a similar line. For all requests whose destination location is pure, all nodes $i \in V_{r^-}$ are deleted and we try to send one unit of flow from an arbitrary start

Table 3
Contribution of valid inequalities (individual results): $n_i = 1$.

Name	Compact			SEC				IPC				CC			
	LB	Time	Gap	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-1-0	100.00	0.04	0.00	90.95	5.58	0.00	65	89.74	13.49	0.00	0	91.61	0.55	0.00	16
5-2-2-1-0	99.99	0.06	0.00	90.57	6.87	0.00	113	89.87	5.58	0.00	0	91.20	5.94	0.00	21
5-2-3-1-0	89.23	0.11	0.00	90.11	10.01	0.00	129	89.39	4.92	0.00	0	91.81	11.51	0.00	16
5-3-1-1-0	87.16	0.23	0.00	85.75	3.35	0.00	315	85.11	11.85	0.00	23	85.59	10.31	0.00	19
5-3-2-1-0	85.79	0.34	0.00	84.86	13.26	0.00	1054	85.57	14.74	0.00	42	84.31	18.65	0.00	16
5-3-3-1-0	86.50	0.55	0.00	84.03	9.03	0.00	805	85.48	30.68	0.00	102	83.81	27.15	0.00	15
6-2-1-1-0	90.00	0.22	0.00	89.89	18.22	0.00	387	89.54	22.40	0.00	8	89.84	19.33	0.00	24
6-2-2-1-0	89.86	0.37	0.00	89.98	12.86	0.00	565	89.70	24.57	0.00	8	89.89	20.28	0.00	29
6-2-3-1-0	89.55	0.43	0.00	89.19	21.47	0.00	950	89.39	30.27	0.00	9	89.19	36.06	0.00	39
6-3-1-1-0	93.30	0.16	0.00	94.58	4.97	0.00	99	93.20	8.25	0.00	0	93.48	10.62	0.00	13
6-3-2-1-0	93.37	0.22	0.00	94.10	8.66	0.00	97	93.51	12.81	0.00	0	93.32	7.64	0.00	13
6-3-3-1-0	92.55	0.28	0.00	93.91	6.59	0.00	299	93.12	9.81	0.00	0	93.10	11.85	0.00	10
7-2-1-1-0	90.08	0.64	0.00	89.89	4.14	0.00	699	89.65	13.41	0.00	9	91.54	7.24	0.00	60
7-2-2-1-0	90.00	0.99	0.00	89.78	19.54	0.00	1011	89.52	40.65	0.00	50	91.52	23.02	0.00	53
7-2-3-1-0	89.75	1.35	0.00	89.45	12.58	0.00	1425	89.06	33.29	0.00	18	91.02	23.63	0.00	56
7-3-1-1-0	86.15	2.01	0.00	85.54	26.10	0.00	1519	85.43	50.13	0.00	75	85.42	48.55	0.00	35
7-3-2-1-0	85.67	2.90	0.00	85.47	31.26	0.00	2385	85.41	74.17	0.00	118	85.41	52.56	0.00	34
7-3-3-1-0	85.32	5.07	0.00	85.43	48.77	0.00	3436	85.66	95.07	0.00	90	85.29	102.87	0.00	43
8-2-1-1-0	83.37	388.97	0.00	81.84	170.02	0.00	3582	82.10	1440.00	5.39	434	85.51	1440.00	2.56	130
8-2-2-1-0	82.23	1440.00	1.65	81.70	688.62	0.00	6632	81.73	1440.00	7.24	988	85.84	1440.00	4.73	122
8-2-3-1-0	82.40	1440.00	3.23	81.42	1440.00	3.39	9430	81.41	1440.00	8.59	1748	85.57	1440.00	5.44	128
8-3-1-1-0	88.71	3.37	0.00	88.70	8.46	0.00	1091	88.53	49.90	0.00	1	88.49	46.58	0.00	81
8-3-2-1-0	88.41	4.49	0.00	88.70	19.13	0.00	1662	88.51	64.54	0.00	0	88.62	69.22	0.00	92
8-3-3-1-0	88.30	5.58	0.00	88.66	17.26	0.00	1791	88.28	111.92	0.00	40	88.56	78.91	0.00	84
9-2-1-1-0	82.64	152.37	0.00	82.15	1294.46	0.00	7384	82.07	1440.00	3.80	408	83.98	1186.90	0.00	108
9-2-2-1-0	82.83	529.50	0.00	82.20	1440.00	7.46	11051	81.96	1440.00	5.82	276	83.59	1440.00	4.83	124
9-2-3-1-0	82.58	1440.00	1.91	82.18	1440.00	5.94	12377	81.95	1440.00	6.63	358	83.84	1440.00	5.77	121
9-3-1-1-0	86.73	8.98	0.00	86.64	134.55	0.00	3600	86.40	219.28	0.00	36	87.90	95.13	0.00	114
9-3-2-1-0	86.90	18.78	0.00	86.52	132.67	0.00	2637	86.42	394.16	0.00	29	87.95	79.86	0.00	79
9-3-3-1-0	87.30	28.76	0.00	86.32	284.81	0.00	4712	86.36	664.29	0.00	328	87.96	277.00	0.00	108
10-2-1-1-0	82.02	1440.00	4.38	82.07	1440.00	5.33	11230	81.85	1440.00	8.53	131	83.18	1440.00	5.90	144
10-2-2-1-0	81.96	1440.00	4.92	82.00	1440.00	6.09	10163	81.85	1440.00	8.53	111	82.97	1440.00	6.11	128
10-2-3-1-0	81.81	1440.00	5.29	81.91	1440.00	8.04	13435	81.85	1440.00	9.98	912	82.97	1440.00	6.89	134
10-3-1-1-0	89.35	19.13	0.00	90.85	46.68	0.00	2007	89.37	354.80	0.00	35	89.85	233.45	0.00	127
10-3-2-1-0	89.29	38.29	0.00	91.01	40.65	0.00	1982	89.24	523.85	0.00	44	89.88	395.22	0.00	124
10-3-3-1-0	89.32	60.63	0.00	90.92	77.52	0.00	3105	89.20	692.24	0.00	133	89.87	387.16	0.00	120
Average	87.79	275.41	3.56	87.20	328.28	6.04	3423	86.87	459.20	7.17	182	87.89	411.31	5.28	72

depot $s \in N^+$ to a node $t \in V_{r+}$. The above described additions of arcs to $\tilde{G}$ are analogously applied. For the weak $\pi-\sigma$-constraints we delete all depots (i.e., all nodes $i \in N$) and try to send one unit of flow from a node $s \in V_{r+}$ to a node $t \in V_{r-}$. Note, that in this case neither r^+ nor r^- need to be pure.

5. Computational Experiments

In this section, we summarize the computational experiments performed to assess the performance of our solution method. The branch-and-cut algorithm is implemented in C++ using CPLEX 12.9 as MIP-solver. All experiments were performed on an Intel Xeon 2.6 GHz CPU with 64 GB of RAM.

As mentioned in Section 1, there are several articles dealing with closely-related problems, most of them testing their implementation on randomly-generated instances adapted to remark special characteristics of the problem or methodology. For example, in the instances from Kerivin et al. (2008) all vehicles are identical, there are no transshipment costs, and there is a completion time limit. Our mathematical formulation in Section 2 is valid also for these specialized instances. Indeed, Fig. 3(a) shows an instance used by Kerivin et al. (2008) to illustrate their problem, for which they report the solution in Fig. 3(b) and our approach generated the solution in Fig. 3(c). However, computational experiments on recreated randomly-generated instances (according to the description in Kerivin et al., 2008) revealed that our implementation has a worse time performance than the reported times in Kerivin et al. (2008). There are several explanations for it. While our formulation is designed to deal with a heterogeneous fleet of vehicles, it suffers symmetry issues when all vehicles are identical. Also, the g variables in our formulation – which allow modelling the transshipment costs in (1) – significantly increase the size of the formulation, but transshipment costs are zero in their instances. Still, the most critical element is the completion time limit $\mathcal{T}$, which strongly helps the approach in Kerivin et al. (2008) when it is small. For these reasons, using the instances in Kerivin et al. (2008) makes a comparison unfair, and we have decided to create a new benchmark collection better adapted to the applications that motivated our research.

The new set of test instances represents generic multimodal transport and less-than-truckload instances. For that purpose, and following the convention in the PDPT literature (see e.g. Cortés et al., 2010; Mitrovic-Minic & Laporte, 2006; Qu & Bard, 2012; Rais et al., 2014), we generated random instances where transshipment is only allowed at dedicated transshipment locations and every customer location corresponds to either the origin or the destination of exactly one request. In all instances, the coordinates of each origin, destination and depot location are randomly chosen in the square $[0, 10000] \times [0, 10000]$ according to a uniform probability distribution. The dedicated transshipment locations are placed at the center of either the customer locations, the origin locations, or the destination locations. For every arc $a \in A$, the travel cost c_a^k and travel time t_a^k are equal to the Euclidean distance between the two locations. In all instances, the number of vehicles is either 2 or 3, with vehicle capacities $q_1 = 100$, $q_2 = 75$ and $q_3 = 50$. The demand of each request is randomly generated with a uniform probability distribution in $[20, 80] \cap \mathbb{N}$. We generated five instances for each combination of number of requests, number of vehicles and

Table 4
Contribution of valid inequalities (individual results): $n_i = 1$.

Name	SCC				PRC				B&C			
	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-1-0	90.68	11.26	0.00	34	90.34	6.48	0.00	55	100.00	0.04	0.00	87
5-2-2-1-0	90.51	4.15	0.00	46	90.29	9.06	0.00	89	96.11	2.96	0.00	130
5-2-3-1-0	90.34	13.89	0.00	36	89.98	13.38	0.00	387	95.52	10.46	0.00	144
5-3-1-1-0	85.58	11.00	0.00	64	92.24	5.68	0.00	249	94.63	2.35	0.00	479
5-3-2-1-0	84.56	19.38	0.00	90	91.94	12.53	0.00	333	93.72	7.05	0.00	614
5-3-3-1-0	83.72	37.47	0.00	78	91.70	20.17	0.00	526	94.41	3.52	0.00	666
6-2-1-1-0	90.78	21.72	0.00	52	89.83	23.74	0.00	399	90.97	9.46	0.00	634
6-2-2-1-0	90.74	16.04	0.00	66	89.92	26.95	0.00	407	90.99	12.44	0.00	703
6-2-3-1-0	89.79	24.92	0.00	61	89.22	35.62	0.00	1031	90.57	11.26	0.00	814
6-3-1-1-0	93.46	4.23	0.00	11	93.80	7.99	0.00	125	99.83	3.79	0.00	130
6-3-2-1-0	93.32	7.97	0.00	17	93.56	7.34	0.00	103	100.00	4.74	0.00	168
6-3-3-1-0	93.13	6.96	0.00	18	93.47	7.08	0.00	112	100.00	3.17	0.00	184
7-2-1-1-0	90.05	10.69	0.00	140	89.67	23.77	0.00	649	93.41	0.94	0.00	285
7-2-2-1-0	90.00	28.29	0.00	194	89.60	48.11	0.00	1047	91.98	10.92	0.00	418
7-2-3-1-0	89.98	46.05	0.00	234	89.23	46.02	0.00	951	93.04	3.03	0.00	608
7-3-1-1-0	85.46	41.60	0.00	103	86.98	23.25	0.00	465	87.52	13.66	0.00	912
7-3-2-1-0	85.51	38.27	0.00	132	86.93	21.58	0.00	674	87.99	14.51	0.00	1112
7-3-3-1-0	85.29	102.90	0.00	142	86.98	32.59	0.00	618	87.70	17.84	0.00	2166
8-2-1-1-0	82.00	1440.00	5.96	1107	81.81	1440.00	7.14	3299	88.82	13.20	0.00	684
8-2-2-1-0	81.68	1440.00	8.96	848	81.67	1440.00	7.93	3475	87.36	20.85	0.00	1582
8-2-3-1-0	81.39	1440.00	8.26	706	81.32	1440.00	8.77	4036	87.76	44.71	0.00	2520
8-3-1-1-0	88.80	35.63	0.00	158	88.48	49.28	0.00	468	89.92	9.04	0.00	1278
8-3-2-1-0	88.85	47.63	0.00	169	88.49	72.58	0.00	713	89.78	6.27	0.00	1142
8-3-3-1-0	88.72	51.31	0.00	152	88.44	106.91	0.00	804	89.88	11.76	0.00	2019
9-2-1-1-0	82.55	1440.00	4.41	1146	82.23	1440.00	6.78	4497	85.87	181.48	0.00	4681
9-2-2-1-0	82.51	1440.00	5.96	716	82.19	1440.00	8.24	5870	85.58	595.45	0.00	7199
9-2-3-1-0	82.27	1440.00	7.11	647	82.14	1440.00	8.80	5771	86.01	685.04	0.00	7936
9-3-1-1-0	88.71	43.99	0.00	210	87.13	334.88	0.00	2094	91.02	18.59	0.00	2163
9-3-2-1-0	88.34	249.03	0.00	349	87.06	620.06	0.00	2191	91.21	16.85	0.00	1860
9-3-3-1-0	87.71	526.05	0.00	359	86.89	1103.92	0.00	3018	90.93	17.35	0.00	1707
10-2-1-1-0	82.97	1440.00	7.31	807	81.81	1440.00	9.09	2631	85.21	944.87	0.00	6959
10-2-2-1-0	82.45	1440.00	8.14	666	81.75	1440.00	9.48	2560	85.13	1440.00	2.67	9073
10-2-3-1-0	82.58	1440.00	8.56	656	81.71	1440.00	10.12	3388	85.15	1440.00	3.63	12221
10-3-1-1-0	89.51	192.69	0.00	295	89.32	255.16	0.00	1152	93.68	12.10	0.00	979
10-3-2-1-0	89.83	458.54	0.00	452	89.45	702.21	0.00	1615	93.25	9.51	0.00	1099
10-3-3-1-0	90.04	468.81	0.00	350	89.36	608.54	0.00	1857	93.15	17.26	0.00	1353
Average	87.33	430.01	7.18	314	87.69	477.36	8.48	1602	91.34	156.01	3.15	2131

number of transshipment locations. From each of these instances, we generated a version with one location – copy ($n_i = 1$), a version with two location-copies ($n_i = 2$) and a version with three location-copies ($n_i = 3$). Finally, from each of these instances, we generated one version with transshipment cost and another version without transshipment cost.

In total, our benchmark collection contains 1080 random instances, available at https://dx.doi.org/10.17632/cp6fjtnw24.1. Each instance is named $a - b - c - d - e$ with the following convention:

- $a \in \{5, 6, 7, 8, 9, 10\}$: number $|R|$ of requests,
- $b \in \{2, 3\}$: number $|K|$ of vehicles,
- $c \in \{1, 2, 3\}$: number $|T|$ of transshipment locations,
- $d \in \{1, 2, 3\}$: number n_i of location-copies,
- $e \in \{0, 1, 2, 3, 4\}$: number to identify the instance.

We use the wild card "$*$" to refer to all instances in a particular category; for example, "$7 - 2 - * - 3 - *$" refers to all instances with seven requests, two vehicles and three location-copies. In all experiments the run time limit is 24 hours for each instance.

In the first set of experiments we evaluate the contribution of the valid inequalities. For that purpose, we selected one instance with transshipment cost from each category and solved it by separately considering each type of inequality (as well as their combination): subtour elimination constraints (SEC) (20), infeasible path constraints (IPC) (25), capacity constraints (CC) (26), strengthened capacity constraints (SCC) (27), and precedence/successor constraints (PRC) (28)–(30). Table 1 shows for each family of valid inequalities, for the combination of all valid inequalities (column *B&C*), and for the execution which uses no valid inequalities (column *Compact*) the following information: the lower bound at the root node expressed as a percentage of the best known upper bound (row *LB*), the time in minutes to solve the instance to optimality (row *Time*), the number of cuts generated (row *# cuts*), the number of instances solved to optimality (row *# solved*), and the percentage gap for instances which could not be solved to optimality (row *Gap*). All numbers represent an average over 36 instances, with the exception of the percentage gap (*Gap*) which is averaged over only the instances which could not be solved to optimality. Tables 3–8 present the results for each instance individually. The results show that regarding the average lower bound there is no significant difference between the individual families of valid inequalities. The worse average lower bound for some families of valid inequalities compared to the execution which uses no valid inequalities is explained by the fact that CPLEX adds additional families of valid inequalities on its own (i.e., clique cuts, cover cuts, flow path cuts, Gomory fractional cuts, generalized upper bound cover cuts, implied bound cuts, lift-and-project cuts, mixed integer rounding cuts and zero-half cuts). With respect to the number of instances solved to optimality, SEC inequalities clearly outperform the other four valid inequalities for $n_i = 1$. For $n_i = 2$ and $n_i = 3$, they perform the worst, though, and SCC inequalities solve the most instances to optimality. The average run time is in large parts a direct consequence of the number of instances solved to optimality. Thus, the results for the average run times

Table 5
Contribution of valid inequalities (individual results): $n_i = 2$.

Name	Compact			SEC				IPC				CC			
	LB	Time	Gap	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-2-0	83.92	3.94	0.00	83.30	698.55	0.00	14695	82.82	132.32	0.00	200	86.23	60.70	0.00	29
5-2-2-2-0	83.52	15.36	0.00	83.80	1440.00	4.41	22991	77.05	461.18	0.00	329	86.43	177.48	0.00	28
5-2-3-2-0	83.67	1090.09	0.00	83.70	1440.00	5.88	24703	83.67	727.63	0.00	467	86.19	241.96	0.00	29
5-3-1-2-0	82.30	1440.00	1.62	82.57	1440.00	4.63	17917	81.47	1440.00	2.83	274	82.26	1317.58	0.00	22
5-3-2-2-0	81.94	1440.00	5.56	80.91	1440.00	8.86	22225	80.38	1440.00	6.41	304	81.33	1440.00	5.62	22
5-3-3-2-0	82.29	1440.00	7.64	80.84	1440.00	9.83	18297	80.23	1440.00	9.17	325	81.13	1440.00	7.45	19
6-2-1-2-0	86.97	1440.00	2.86	87.27	1440.00	6.12	19001	86.86	1440.00	0.72	440	87.35	1440.00	1.63	51
6-2-2-2-0	87.24	1440.00	6.56	86.50	1440.00	6.30	21045	86.67	1440.00	4.70	137	86.19	1440.00	4.68	43
6-2-3-2-0	86.76	1440.00	3.93	86.26	1440.00	8.31	22827	86.07	1440.00	6.61	136	86.01	1440.00	6.25	46
6-3-1-2-0	87.21	1440.00	1.77	87.16	293.44	0.00	4540	86.25	409.06	0.00	146	87.84	177.15	0.00	24
6-3-2-2-0	87.07	1440.00	2.82	83.38	1440.00	1.92	11294	86.80	1440.00	1.45	83	83.40	938.75	0.00	28
6-3-3-2-0	87.12	1440.00	6.26	87.15	1440.00	4.13	11963	87.09	1440.00	2.53	168	87.99	1440.00	2.01	24
7-2-1-2-0	86.13	1440.00	5.95	85.96	1440.00	9.98	20460	85.81	1440.00	8.09	259	88.28	1440.00	5.03	78
7-2-2-2-0	79.91	1440.00	5.70	79.96	1440.00	14.43	30206	86.01	1440.00	8.15	77	86.09	1440.00	6.24	77
7-2-3-2-0	86.08	1440.00	6.83	85.82	1440.00	11.02	22237	85.77	1440.00	9.18	78	88.36	1440.00	6.83	75
7-3-1-2-0	77.95	1440.00	8.82	78.14	1440.00	15.27	25363	77.55	1440.00	14.07	685	77.92	1440.00	14.01	49
7-3-2-2-0	78.05	1440.00	11.81	78.33	1440.00	17.32	27574	77.92	1440.00	15.31	374	77.91	1440.00	15.42	48
7-3-3-2-0	78.53	1440.00	10.04	79.20	1440.00	16.70	24298	77.61	1440.00	16.39	204	78.94	1440.00	14.59	38
8-2-1-2-0	80.23	1440.00	14.36	80.18	1440.00	17.71	22703	80.05	1440.00	16.06	284	85.88	1440.00	10.51	127
8-2-2-2-0	82.31	1440.00	13.46	82.06	1440.00	16.97	23745	82.08	1440.00	14.60	257	86.23	1440.00	10.67	126
8-2-3-2-0	82.24	1440.00	13.33	82.19	1440.00	16.90	24656	81.99	1440.00	15.26	178	86.53	1440.00	10.53	110
8-3-1-2-0	81.84	1440.00	9.70	82.01	1440.00	14.81	16665	82.24	1440.00	12.05	140	83.29	1440.00	11.46	99
8-3-2-2-0	81.54	1440.00	9.24	81.54	1440.00	15.50	23858	82.04	1440.00	13.42	204	82.99	1440.00	12.28	99
8-3-3-2-0	82.21	1440.00	11.18	81.92	1440.00	15.52	26058	82.06	1440.00	13.93	43	83.21	1440.00	12.61	86
9-2-1-2-0	81.40	1440.00	12.13	81.44	1440.00	16.16	15642	81.28	1440.00	13.43	165	83.92	1440.00	12.47	98
9-2-2-2-0	73.72	1440.00	14.51	73.72	1440.00	23.83	7547	81.10	1440.00	15.31	671	78.49	1440.00	15.40	98
9-2-3-2-0	81.24	1440.00	13.56	80.86	1440.00	16.49	23564	80.95	1440.00	14.95	339	83.61	1440.00	12.91	103
9-3-1-2-0	81.75	1440.00	10.67	81.94	1440.00	14.56	25058	81.82	1440.00	13.19	110	84.12	1440.00	12.12	115
9-3-2-2-0	82.98	1440.00	9.99	83.04	1440.00	12.90	22603	82.71	1440.00	13.09	159	85.20	1440.00	12.30	113
9-3-3-2-0	81.93	1440.00	12.08	81.89	1440.00	14.98	24839	81.29	1440.00	15.11	247	83.97	1440.00	12.69	103
10-2-1-2-0	80.93	1440.00	14.83	80.91	1440.00	18.30	20916	80.60	1440.00	17.33	806	83.78	1440.00	14.42	124
10-2-2-2-0	74.83	1440.00	16.01	74.83	1440.00	23.54	31330	80.69	1440.00	17.31	215	80.01	1440.00	15.60	125
10-2-3-2-0	80.79	1440.00	15.56	80.94	1440.00	18.23	23469	80.93	1440.00	17.05	297	83.92	1440.00	14.19	104
10-3-1-2-0	84.46	1440.00	9.64	84.69	1440.00	13.53	15564	84.51	1440.00	12.42	119	85.96	1440.00	10.98	111
10-3-2-2-0	84.86	1440.00	10.18	84.64	1440.00	13.70	15123	84.38	1440.00	12.51	156	85.69	1440.00	10.69	102
10-3-3-2-0	84.58	1440.00	10.48	84.74	1440.00	13.63	17770	84.49	1440.00	12.46	199	86.18	1440.00	11.20	106
Average	82.51	1350.82	9.37	82.33	1387.56	13.01	20632	82.53	1328.06	11.41	258	84.24	1280.93	10.43	74

depict an almost identical picture. While the average gap for instances which were not solved to optimality is again more homogeneous between the individual valid inequalities, the average number of generated cuts varies significantly from less than one hundred for CC inequalities to several thousand for SEC inequalities. Compared to each individual family of inequalities, and none of them, combining all five families of inequalities significantly reduces the computational time, improves the lower bound, and reduces the gap for instances which were not solved to optimality. In the following, all experiments are performed using the five families of inequalities.

It is worth mentioning here that we also tried an alternative variant of the branch-and-cut algorithm which uses Benders' Decomposition. The motivation was to deal with an integer linear program without the f and g variables, which are responsible for the large size of the compact formulation (1)–(15). To solve the alternative variant we decomposed the compact formulation in a master problem (MP) and a subproblem (SP). The MP contained variables x, y, z, e and s along with constraints (2)–(4), (9), (11) and (13)–(14), while the SP contained the continuous variables f and g together with constraints (5)–(8), (10) and (12). While this variant outperformed the proposed algorithm on some instances, overall it did not yield a better performance.

In a second set of experiments we measure the impact of both transshipment costs and the value of n_i (number of location-copies) on the performance of the branch-and-cut algorithm. Table 2 shows the average computing time in minutes (column *Time*) of solving the instances once without transshipment costs (column *B&C*) and once with transshipment costs (column *B&C-WTC*) for different values of n_i. The results show that considering transshipment costs can significantly improve the performance of the algorithm. The average computation time varies between approximately 21% less and 4% more than when transshipment costs are not considered.

Table 2 also shows that n_i has a significant impact on the performance of the algorithm. The larger n_i the worse the performance. For $n_i = 1$ we were able to solve 336 out of 380 instances to optimality within the run time limit. When using $n_i = 2$ the number of instances solved to optimality reduces to 107, and for $n_i = 3$ it further reduces to 20. The reason for this performance loss is twofold. First, it is due to the induced increase in the number of variables, respectively constraints. Second, having more than one vertex copy in the graph also significantly increases the symmetry in the model. In the course of preliminary experiments we tested symmetry-breaking constraints, both for location-copies and for vehicles, to alleviate the problem. However, the effects were only minimal, which is why we are not using any such constraints in the final form of our branch-and-cut algorithm.

For $n_i = 2$ and $n_i = 3$, Table 2 additionally shows the average percentage improvement of the objective value of the respective problem compared to the objective value of the problem with $n_i = 1$ (column *Improv.*). The percentage improvement was calculated as $100(UB_{n_i=1} - UB_{n_i=m})/UB_{n_i=1}$, where *UB* denotes the upper bound of the objective value and $m \in \{2, 3\}$. A value for improvement was calculated only when the problem with $n_i = 1$ was solved to optimality and a valid upper bound was found for

Table 6
Contribution of valid inequalities (individual results): $n_i = 2$.

Name	SCC				PRC				B&C			
	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-2-0	86.81	65.77	0.00	127	86.69	190.62	0.00	3892	92.05	25.71	0.00	1512
5-2-2-2-0	86.38	100.13	0.00	151	87.04	493.30	0.00	4896	90.25	48.00	0.00	4273
5-2-3-2-0	86.17	213.25	0.00	121	86.87	1440.00	1.83	9997	91.00	122.80	0.00	6334
5-3-1-2-0	84.22	812.28	0.00	208	91.31	111.93	0.00	2360	93.77	62.97	0.00	3248
5-3-2-2-0	82.25	1440.00	0.13	199	91.15	946.53	0.00	7472	93.75	232.76	0.00	9497
5-3-3-2-0	80.98	1440.00	5.05	161	91.11	1440.00	2.79	8100	93.77	1440.00	3.35	18867
6-2-1-2-0	88.75	745.75	0.00	380	87.73	1440.00	2.23	9509	89.91	960.78	0.00	14880
6-2-2-2-0	87.29	1440.00	1.25	356	87.57	1440.00	6.15	9198	89.35	1440.00	5.43	14072
6-2-3-2-0	86.58	1440.00	4.02	299	87.62	1440.00	6.64	10645	89.49	1440.00	3.67	15178
6-3-1-2-0	87.26	173.54	0.00	111	90.40	68.31	0.00	1310	91.80	12.16	0.00	1449
6-3-2-2-0	84.55	643.86	0.00	182	88.56	63.13	0.00	905	91.87	15.39	0.00	1307
6-3-3-2-0	87.52	1440.00	2.68	131	90.61	111.84	0.00	1420	91.93	15.65	0.00	1565
7-2-1-2-0	88.47	1440.00	4.81	641	86.51	1440.00	9.54	10159	90.48	1440.00	3.29	17183
7-2-2-2-0	85.49	1440.00	5.08	741	80.38	1440.00	10.21	13196	90.80	1440.00	5.71	16472
7-2-3-2-0	88.34	1440.00	5.90	594	86.33	1440.00	10.41	13551	90.33	1440.00	5.08	17693
7-3-1-2-0	79.92	1440.00	11.24	391	82.30	1440.00	9.22	3875	83.73	1440.00	6.25	15443
7-3-2-2-0	81.34	1440.00	12.53	322	82.39	1440.00	10.86	4728	83.92	1440.00	6.48	17488
7-3-3-2-0	80.76	1440.00	12.60	287	82.76	1440.00	10.48	4097	84.14	1440.00	11.27	18049
8-2-1-2-0	80.95	1440.00	15.61	693	80.26	1440.00	17.40	8590	89.00	1440.00	8.55	15447
8-2-2-2-0	82.39	1440.00	14.85	583	82.29	1440.00	15.85	9119	89.37	1440.00	9.53	14096
8-2-3-2-0	82.49	1440.00	15.16	458	82.46	1440.00	15.70	8664	89.34	1440.00	9.61	16339
8-3-1-2-0	85.95	1440.00	9.34	426	82.35	1440.00	13.50	4822	85.60	1440.00	9.42	19114
8-3-2-2-0	85.16	1440.00	10.38	450	81.96	1440.00	13.79	4595	87.28	1440.00	9.55	18562
8-3-3-2-0	85.78	1440.00	10.69	391	82.31	1440.00	15.26	6181	86.62	1440.00	10.54	18216
9-2-1-2-0	83.07	1440.00	12.91	650	82.30	1440.00	14.56	8527	85.68	1440.00	13.19	7890
9-2-2-2-0	76.60	1440.00	15.74	580	76.18	1440.00	17.31	7943	85.83	1440.00	14.14	4741
9-2-3-2-0	82.16	1440.00	13.26	466	81.79	1440.00	15.67	10437	85.49	1440.00	12.86	19619
9-3-1-2-0	84.41	1440.00	10.24	488	84.81	1440.00	11.35	4511	89.42	1440.00	7.81	17589
9-3-2-2-0	85.64	1440.00	9.10	368	86.04	1440.00	11.56	5501	90.02	1440.00	9.81	5506
9-3-3-2-0	84.24	1440.00	10.89	333	84.71	1440.00	13.11	6882	89.07	1440.00	8.93	21280
10-2-1-2-0	84.95	1440.00	13.04	612	81.21	1440.00	17.45	8997	86.24	1440.00	13.68	4731
10-2-2-2-0	80.92	1440.00	14.81	739	76.04	1440.00	19.82	6919	86.37	1440.00	12.94	19035
10-2-3-2-0	83.92	1440.00	13.20	529	81.16	1440.00	17.41	9025	86.42	1440.00	12.76	13341
10-3-1-2-0	87.80	1440.00	8.82	474	85.12	1440.00	12.26	3109	89.76	1440.00	8.26	9646
10-3-2-2-0	87.40	1440.00	9.57	425	85.15	1440.00	12.47	4365	90.00	1440.00	8.30	12291
10-3-3-2-0	87.46	1440.00	9.68	391	85.12	1440.00	12.06	3783	89.56	1440.00	9.20	8970
Average	84.57	1236.52	9.74	402	84.68	1215.16	11.96	6702	88.98	1121.56	8.87	12248

the problem with $n_i > 1$. A negative value indicates that $UB_{n_i=m} > UB_{n_i=1}$, which can happen when the problem with $n_i > 1$ was not solved to optimality. A "-" indicates that no value for improvement could be calculated; either because the problem with $n_i = 1$ was not solved to optimality, or because no valid upper bound was found for the problem with $n_i > 1$. The results show that for smaller instances with $n_i = 2$ the average percentage improvement is roughly 1–2%, on average. For larger instances, and for $n_i = 3$, the average percentage improvement is on average negative, meaning worse upper bounds were found for the problem with $n_i > 1$ than for the problem with $n_i = 1$. Depending on whether transshipment costs are considered and the value of n_i, the average percentage deterioration is between 7–20%, on average. The negative values and the smaller values for $n_i = 3$ than for $n_i = 2$ are explained by the increased difficulty of solving problems with larger n_i and the consequent (potentially) worse upper bound in case the time limit is reached.

Finally, as shown in Fig. 2, (significant) cost reductions can be achieved by considering split loads and transshipments simultaneously, compared to considering only split loads, only transshipments, or neither. The amount of achievable cost savings depends significantly on the geographical distribution of the customers (i.e. their coordinates) in combination with their demands; which are all randomly generated values in our instances. The cost structure and capacity of the involved vehicles, and the magnitude of the transshipment cost, have a large influence as well; which are, again, arbitrarily chosen values in our instances. Unfortunately, this led to only few of our instances needing split loads and/or transshipments in the optimal solution. It is generally not trivial to assess which prerequisites must be given in an instance so that split loads and/or transshipments are beneficial. A comprehensive analysis regarding favorable circumstances for split loads and/or transshipments is out of scope of this article (see Nowak et al. (2008) for a related analysis regarding split loads). Still, it is a-priori unknown whether split loads and transshipments will allow a better solution for an instance. Hence, the algorithm described and evaluated in this section may be useful, either to find the optimal solution, or to find a near-optimal solution if interrupted by a time limit.

6. Conclusion

This article presents a new arc-based mixed-integer formulation for the Pickup and Delivery Problem with split loads and transshipments. We have shown through an example that the PDPSLT can yield solutions which are cheaper than those obtained from the PDPT, the PDPSL, or the PDP. The article also describes a branch-and-cut algorithm for this new formulation and presents computational experiments which show its effectiveness. To the best of our knowledge, this is the first work which solves instances with eight requests and three transshipment locations to optimality, both for the PDPT and the PDPSLT.

Table 7
Contribution of valid inequalities (individual results): $n_i = 3$.

Name	Compact				SEC				IPC				CC			
	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-3-0	76.60	62.38	0.00	0	76.60	1440.00	13.15	50033	76.60	818.86	0.00	351	82.21	472.50	0.00	35
5-2-2-3-0	76.60	248.74	0.00	0	76.60	1440.00	14.65	55953	76.60	1440.00	5.16	255	82.21	815.24	0.00	29
5-2-3-3-0	76.93	495.67	0.00	0	76.60	1440.00	16.23	43317	76.60	1440.00	6.91	155	82.23	1440.00	3.12	32
5-3-1-3-0	77.22	522.90	0.00	0	76.84	1440.00	11.23	63500	77.24	1440.00	6.96	103	78.52	1440.00	6.49	24
5-3-2-3-0	76.92	1440.00	4.64	0	77.09	1440.00	15.02	62473	76.84	1440.00	9.24	42	78.64	1440.00	7.19	18
5-3-3-3-0	76.84	1440.00	5.43	0	77.79	1440.00	10.88	64560	77.84	1440.00	12.22	100	78.52	1440.00	9.95	20
6-2-1-3-0	79.80	1440.00	1.82	0	79.84	1440.00	10.89	54617	79.65	1440.00	9.55	147	79.81	1440.00	7.37	38
6-2-2-3-0	80.58	1440.00	3.41	0	79.58	1440.00	12.32	63204	80.03	1440.00	9.46	165	79.54	1440.00	9.13	38
6-2-3-3-0	79.67	1440.00	5.00	0	79.67	1440.00	15.17	64042	79.54	1440.00	10.76	70	79.67	1440.00	10.40	41
6-3-1-3-0	81.61	216.95	0.00	0	80.75	1440.00	11.18	53824	80.75	1440.00	6.51	153	82.74	1440.00	3.18	31
6-3-2-3-0	80.75	1368.27	0.00	0	80.75	1440.00	13.02	49075	83.29	1440.00	8.30	52	82.74	1440.00	7.25	32
6-3-3-3-0	80.78	1440.00	2.65	0	80.78	1440.00	12.61	47518	82.62	1440.00	8.33	102	82.77	1440.00	7.98	27
7-2-1-3-0	82.43	1440.00	7.76	0	82.43	1440.00	14.24	51357	80.05	1440.00	11.59	1140	87.20	1440.00	7.08	67
7-2-2-3-0	79.91	1440.00	8.79	0	79.93	1440.00	16.79	66119	79.91	1440.00	12.19	160	86.09	1440.00	8.58	71
7-2-3-3-0	79.91	1440.00	9.95	0	79.92	1440.00	17.71	71835	79.91	1440.00	13.44	424	85.83	1440.00	9.04	71
7-3-1-3-0	68.38	1440.00	15.28	0	68.38	1440.00	25.52	71089	68.37	1440.00	20.68	591	68.71	1440.00	22.06	44
7-3-2-3-0	68.38	1440.00	15.68	0	68.38	1440.00	26.38	74019	68.38	1440.00	23.80	330	68.71	1440.00	21.27	40
7-3-3-3-0	68.38	1440.00	17.43	0	68.38	1440.00	28.50	72136	68.38	1440.00	23.20	287	68.71	1440.00	23.82	32
8-2-1-3-0	77.47	1440.00	15.68	0	77.66	1440.00	20.20	55698	76.15	1440.00	18.56	334	85.24	1440.00	11.28	117
8-2-2-3-0	75.09	1440.00	18.17	0	75.09	1440.00	21.91	62973	75.09	1440.00	20.68	186	80.03	1440.00	14.30	109
8-2-3-3-0	77.64	1440.00	15.73	0	77.64	1440.00	19.39	61056	77.64	1440.00	18.52	298	84.80	1440.00	11.55	105
8-3-1-3-0	76.23	1440.00	13.63	0	76.23	1440.00	22.23	56362	79.38	1440.00	16.89	139	78.39	1440.00	17.05	92
8-3-2-3-0	76.23	1440.00	14.29	0	76.23	1440.00	21.78	55084	76.23	1440.00	19.15	67	78.36	1440.00	16.86	94
8-3-3-3-0	79.40	1440.00	14.36	0	76.23	1440.00	21.43	75780	76.23	1440.00	20.47	52	78.37	1440.00	17.90	87
9-2-1-3-0	73.77	1440.00	16.22	0	73.77	1440.00	25.21	83106	73.77	1440.00	20.73	61	78.46	1440.00	17.37	84
9-2-2-3-0	73.77	1440.00	16.94	0	73.78	1440.00	24.72	119287	73.77	1440.00	21.51	79	78.50	1440.00	17.76	81
9-2-3-3-0	73.43	1440.00	17.87	0	73.43	1440.00	25.44	49824	73.25	1440.00	22.00	107	78.10	1440.00	18.47	78
9-3-1-3-0	77.67	1440.00	17.20	0	76.32	1440.00	22.39	41893	77.78	1440.00	20.29	67	79.16	1440.00	17.82	92
9-3-2-3-0	80.86	1440.00	14.00	0	80.75	1440.00	18.34	57218	79.41	1440.00	17.65	53	82.52	1440.00	13.57	83
9-3-3-3-0	78.33	1440.00	15.74	0	78.33	1440.00	20.27	39148	78.33	1440.00	19.19	21	81.33	1440.00	16.05	69
10-2-1-3-0	79.87	1440.00	16.74	0	79.78	1440.00	20.06	48800	74.83	1440.00	21.13	56	82.41	1440.00	15.33	96
10-2-2-3-0	74.84	1440.00	18.12	0	76.49	1440.00	22.61	42837	78.63	1440.00	19.17	124	80.81	1440.00	16.93	95
10-2-3-3-0	76.32	1440.00	17.52	0	79.42	1440.00	20.27	36396	74.83	1440.00	22.03	186	82.34	1440.00	15.51	85
10-3-1-3-0	81.65	1440.00	13.12	0	81.83	1440.00	16.67	21393	75.30	1440.00	19.50	74	81.90	1440.00	14.33	77
10-3-2-3-0	75.36	1440.00	16.11	0	75.37	1440.00	20.84	16679	79.57	1440.00	17.16	135	78.85	1440.00	16.82	89
10-3-3-3-0	57.04	1440.00	37.86	0	57.04	1440.00	42.78	20481	61.26	1440.00	36.81	62	59.60	1440.00	37.29	88
Average	76.57	1280.97	13.57	0	76.55	1440.00	19.22	56186	76.50	1422.75	16.28	187	79.56	1395.77	13.83	64

Table 8
Contribution of valid inequalities (individual results): $n_i = 3$.

Name	SCC				PRC				B&C			
	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
5-2-1-3-0	84.26	258.85	0.00	225	83.04	1440.00	6.93	24276	90.38	1440.00	5.54	9836
5-2-2-3-0	83.92	911.96	0.00	247	83.04	1440.00	8.85	23298	90.12	1440.00	7.49	10639
5-2-3-3-0	84.27	1440.00	1.64	233	83.06	1440.00	12.33	25785	90.40	1440.00	7.16	19230
5-3-1-3-0	80.91	1440.00	6.51	200	90.97	1440.00	5.27	12559	93.71	1440.00	3.63	18627
5-3-2-3-0	80.90	1440.00	6.86	158	90.97	1440.00	6.32	16064	93.25	1440.00	4.75	22581
5-3-3-3-0	81.62	1440.00	8.53	176	90.97	1440.00	7.82	14826	93.71	1440.00	4.71	16343
6-2-1-3-0	81.62	1440.00	5.81	408	86.88	1440.00	11.59	21680	89.70	1440.00	10.05	23709
6-2-2-3-0	80.99	1440.00	8.65	363	86.88	1440.00	11.06	23584	88.94	1440.00	10.83	19921
6-2-3-3-0	80.92	1440.00	9.45	306	86.93	1440.00	12.62	24913	89.01	1440.00	10.86	21820
6-3-1-3-0	83.45	1052.14	0.00	225	88.18	1440.00	7.28	18092	91.03	1440.00	3.46	9251
6-3-2-3-0	82.42	1440.00	7.61	250	88.19	1440.00	6.32	11083	91.43	1440.00	1.30	16389
6-3-3-3-0	82.42	1440.00	7.40	208	88.18	1440.00	7.16	14429	91.43	1440.00	5.12	7568
7-2-1-3-0	87.70	1440.00	6.67	551	83.26	1440.00	13.08	24620	89.33	1440.00	8.89	27864
7-2-2-3-0	84.99	1440.00	8.80	542	80.38	1440.00	16.75	28942	90.44	1440.00	8.16	25871
7-2-3-3-0	85.48	1440.00	9.22	551	80.38	1440.00	17.50	34500	90.05	1440.00	8.92	12784
7-3-1-3-0	78.61	1440.00	17.33	287	80.04	1440.00	18.20	12958	82.89	1440.00	14.72	8983
7-3-2-3-0	78.36	1440.00	17.10	245	80.03	1440.00	18.50	13523	82.89	1440.00	14.72	12081
7-3-3-3-0	78.40	1440.00	16.92	232	80.03	1440.00	18.37	8461	82.89	1440.00	14.72	11381
8-2-1-3-0	79.36	1440.00	17.01	552	79.53	1440.00	19.59	29577	89.14	1440.00	10.47	23569
8-2-2-3-0	76.67	1440.00	20.03	585	78.07	1440.00	21.72	28861	84.99	1440.00	13.35	20357
8-2-3-3-0	79.10	1440.00	17.22	471	80.64	1440.00	19.11	31196	89.43	1440.00	10.46	17962
8-3-1-3-0	83.70	1440.00	13.01	405	77.89	1440.00	20.72	18493	86.07	1440.00	11.71	15630
8-3-2-3-0	83.97	1440.00	12.97	408	77.92	1440.00	21.30	18474	85.37	1440.00	13.85	19862
8-3-3-3-0	83.69	1440.00	13.07	363	77.89	1440.00	21.14	21504	85.16	1440.00	14.12	27832
9-2-1-3-0	78.29	1440.00	17.66	513	76.21	1440.00	21.85	27881	83.28	1440.00	15.86	19936
9-2-2-3-0	76.86	1440.00	17.29	372	76.22	1440.00	21.72	21460	83.74	1440.00	15.59	7822
9-2-3-3-0	77.39	1440.00	19.41	368	75.94	1440.00	22.97	22559	83.14	1440.00	16.86	12675

(*continued on next page*)

Table 8 (*continued*)

Name	SCC				PRC				B&C			
	LB	Time	Gap	#	LB	Time	Gap	#	LB	Time	Gap	#
9-3-1-3-0	80.70	1440.00	16.78	291	79.63	1440.00	19.26	18110	86.24	1440.00	13.76	14374
9-3-2-3-0	83.89	1440.00	13.44	263	83.67	1440.00	15.96	14780	89.14	1440.00	10.17	17554
9-3-3-3-0	82.42	1440.00	15.35	284	81.73	1440.00	17.67	13008	87.16	1440.00	12.64	21304
10-2-1-3-0	83.81	1440.00	14.08	395	80.22	1440.00	19.47	19066	84.50	1440.00	15.37	18228
10-2-2-3-0	82.38	1440.00	15.34	462	77.22	1440.00	22.61	19107	85.60	1440.00	14.39	12840
10-2-3-3-0	83.51	1440.00	14.38	312	79.96	1440.00	19.96	13727	84.76	1440.00	15.15	16469
10-3-1-3-0	86.00	1440.00	10.79	314	83.63	1440.00	14.12	7695	89.01	1440.00	10.26	13258
10-3-2-3-0	84.52	1440.00	13.31	329	80.34	1440.00	17.51	11064	89.09	1440.00	9.62	9969
10-3-3-3-0	63.69	1440.00	36.02	241	60.81	1440.00	38.16	6393	67.44	1440.00	31.73	8916
Average	81.42	1381.75	13.20	343	81.64	1440.00	16.13	19349	87.36	1440.00	11.12	16484

Nevertheless, in order to improve the significance of analyses regarding the benefits of the PDPSLT, and its potential when applying it in practice, solving larger instances to optimality is imperative. Since the main constraining factor in our formulation is the need of location-copies, a promising direction for further research is the design of set-partitioning (set-covering) formulations. In such formulations, the constraints governing the several-visits to a location by the same vehicle aspect could be moved to the sub-problem where they should be handled more efficiently. The main challenge will be the time synchronization at transshipment locations, which will need to be handled in the master problem.

In order to gain more practical insights, another research line of interest would be an analysis of the benefit of load splits and transshipments on a collection of instances from real-world data. Although it is likely that the observed benefits cannot be generalized, they would still provide an indication of how much can be gained in practice. The computational experiments in this article make evident that such an analysis will need to be conducted by means of a (meta)heuristic solution approach, capable of handling real-world sized instances.

Acknowledgments

The financial support by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development and the Christian Doppler Research Association is gratefully acknowledged. This work was partially supported by the research projects ProID2017010132 (Gobierno de Canarias) and MTM2015-63680-R (MINECO/FEDER). We would also like to thank the five anonymous referees for their helpful comments.

References

Applegate, D., Bixby, R., Chvátal, V., & Cook, W. (2003). Concorde tsp solver. http://www.math.uwaterloo.ca/tsp/concorde.html.

Archetti, C., Bianchessi, N., & Speranza, M. G. (2011a). A column generation approach for the split delivery vehicle routing problem. *Networks, 58*(4), 241–254.

Archetti, C., Bouchard, M., & Desaulniers, G. (2011b). Enhanced branch and price and cut for vehicle routing with split deliveries and time windows. *Transportation Science, 45*(3), 285–298.

Archetti, C., & Speranza, M. G. (2008). The split delivery vehicle routing problem: A survey. In B. Golden, S. Raghavan, & E. Wasil (Eds.), *The vehicle routing problem: Latest advances and new challenges* (pp. 103–122). Boston, MA: Springer US.

Archetti, C., & Speranza, M. G. (2012). Vehicle routing problems with split deliveries. *International transactions in operational research, 19*(1-2), 3–22.

Ascheuer, N., Fischetti, M., & Grötschel, M. (2000). A polyhedral study of the asymmetric traveling salesman problem with time windows. *Networks, 36*(2), 69–79.

Balas, E., Fischetti, M., & Pulleyblank, W. R. (1995). The precedence-constrained asymmetric traveling salesman polytope. *Mathematical programming, 68*(1-3), 241–265.

Berbeglia, G., Cordeau, J.-F., Gribkovskaia, I., & Laporte, G. (2007). Static pickup and delivery problems: a classification scheme and survey. *Top, 15*(1), 1–31.

Bouros, P., Sacharidis, D., Dalamagas, T., & Sellis, T. (2011). Dynamic pickup and delivery with transfers. In D. Pfoser, Y. Tao, K. Mouratidis, M. A. Nascimento, M. Mokbel, S. Shekhar, & Y. Huang (Eds.), *Advances in spatial and temporal databases* (pp. 112–129). Berlin, Heidelberg: Springer Berlin Heidelberg.

Casazza, M., Ceselli, A., & Calvo, R. W. (2020). A route decomposition approach for the single commodity split pickup and split delivery vehicle routing problem. *European Journal of Operational Research*, Inpress.

Cordeau, J.-F. (2006). A branch-and-cut algorithm for the dial-a-ride problem. *Operations Research, 54*(3), 573–586.

Cortés, C. E., Matamala, M., & Contardo, C. (2010). The pickup and delivery problem with transfers: Formulation and a branch-and-cut solution method. *European Journal of Operational Research, 200*(3), 711–724.

Desaulniers, G. (2010). Branch-and-price-and-cut for the split-delivery vehicle routing problem with time windows. *Operations research, 58*(1), 179–192.

Dror, M., Laporte, G., & Trudeau, P. (1994). Vehicle routing with split deliveries. *Discrete Applied Mathematics, 50*(3), 239–254.

Dror, M., & Trudeau, P. (1989). Savings by split delivery routing. *Transportation Science, 23*(2), 141–145.

Dror, M., & Trudeau, P. (1990). Split delivery routing. *Naval Research Logistics, 37*(3), 383–402.

Gendreau, M., Dejax, P., Feillet, D., & Gueguen, C. (2006). Vehicle routing with time windows and split deliveries. *Technical Report*. Laboratoire Informatique dAvignon, Avignon, France.

Grangier, P., Gendreau, M., Lehuédé, F., & Rousseau, L.-M. (2017). A matheuristic based on large neighborhood search for the vehicle routing problem with cross–docking. *Computers & Operations Research, 84*, 116–126.

Guastaroba, G., Speranza, M. G., & Vigo, D. (2016). Intermediate facilities in freight transportation planning: a survey. *Transportation Science, 50*(3), 763–789.

Haddad, M. N., Martinelli, R., Vidal, T., Martins, S., Ochi, L. S., Souza, M. J. F., & Hartl, R. (2018). Large neighborhood-based metaheuristic and branch-and-price for the pickup and delivery problem with split loads. *European Journal of Operational Research, 270*, 1014–1027.

Kerivin, H., Lacroix, M., Mahjoub, A. R., & Quilliot, A. (2008). The splittable pickup and delivery problem with reloads. *European Journal of Industrial Engineering, 2*(2), 112–133.

Maknoon, Y., & Laporte, G. (2017). Vehicle routing with cross-dock selection. *Computers & Operations Research, 77*, 254–266.

Masson, R., Lehuédé, F., & Péton, O. (2013). An adaptive large neighborhood search for the pickup and delivery problem with transfers. *Transportation Science, 47*(3), 344–355.

Masson, R., Lehuédé, F., & Péton, O. (2014a). The dial-a-ride problem with transfers. *Computers & Operations Research, 41*, 12–23.

Masson, R., Ropke, S., Lehuédé, F., & Péton, O. (2014b). A branch-and-cut-and-price approach for the pickup and delivery problem with shuttle routes. *European Journal of Operational Research, 236*(3), 849–862.

Mitchell, J. E. (2011). Branch and cut. In J. J. Cochran, L. A. Cox Jr., P. Keskinocak, J. P. Kharoufeh, & J. C. Smith (Eds.), *Wiley encyclopedia of operations research and management science*. Wiley.

Mitrovic-Minic, S., & Laporte, G. (2006). The pickup and delivery problem with time windows and transshipment. *Infor, 44*(3), 217.

Naddef, D., & Rinaldi, G. (2002). Branch-and-cut algorithms for the capacitated vrp. In P. Toth, & D. Vigo (Eds.), *The vehicle routing problem* (pp. 53–84). SIAM.

Neves-Moreira, F., Amorim, P., Guimarães, L., & Almada-Lobo, B. (2016). A long-haul freight transportation problem: Synchronizing resources to deliver requests passing through multiple transshipment locations. *European Journal of Operational Research, 248*(2), 487–506.

Nowak, M., Ergun, Ö., & White III, C. C. (2008). Pickup and delivery with split loads. *Transportation Science, 42*(1), 32–43.

Nowak, M., Ergun, O., & White III, C. C. (2009). An empirical study on the benefit of split loads with the pickup and delivery problem. *European Journal of Operational Research, 198*(3), 734–740.

Parragh, S. N., Doerner, K. F., & Hartl, R. F. (2008). A survey on pickup and delivery problems. *Journal für Betriebswirtschaft, 58*(1), 21–51.

Parragh, S. N., Pinho de Sousa, J., & Almada-Lobo, B. (2015). The dial-a-ride problem with split requests and profits. *Transportation Science, 49*(2), 311–334.

Petersen, H. L., & Ropke, S. (2011). The pickup and delivery problem with cross–docking opportunity. In J. W. Böse, H. Hu, C. Jahn, X. Shi, R. Stahlbock, & S. Voß (Eds.), *Computational logistics* (pp. 101–113). Berlin, Heidelberg: Springer Berlin Heidelberg.

Qu, Y., & Bard, J. F. (2012). A grasp with adaptive large neighborhood search for pickup and delivery problems with transshipment. *Computers & Operations Research, 39*(10), 2439–2456.
Rais, A., Alvelos, F., & Carvalho, M. S. (2014). New mixed integer-programming model for the pickup-and-delivery problem with transshipment. *European Journal of Operational Research, 235*(3), 530–539.
Ropke, S., Cordeau, J.-F., & Laporte, G. (2007). Models and branch-and-cut algorithms for pickup and delivery problems with time windows. *Networks, 49*(4), 258–272.
Şahin, M., Çavuşlar, G., Öncan, T., Şahin, G., & Aksu, D. T. (2013). An efficient heuristic for the multi-vehicle one-to-one pickup and delivery problem with split loads. *Transportation Research Part C: Emerging Technologies, 27*, 169–188.
Stålhane, M., Andersson, H., Christiansen, M., Cordeau, J.-F., & Desaulniers, G. (2012). A branch-price-and-cut method for a ship routing and scheduling problem with split loads. *Computers & Operations Research, 39*(12), 3361–3375.
Toth, P., & Vigo, D. (2014). *Vehicle routing: problems, methods, and applications*. SIAM.
Wolsey, L. A. (1998). *Integer Programming*. Wiley.